



PDF Download
3654440.pdf
30 December 2025
Total Citations: 5
Total Downloads:
1084

Latest updates: <https://dl.acm.org/doi/10.1145/3654440>

RESEARCH-ARTICLE

DiPri: Distance-Based Seed Prioritization for Greybox Fuzzing

RUIXIANG QIAN, Nanjing University, Nanjing, Jiangsu, China

QUANJUN ZHANG, Nanjing University, Nanjing, Jiangsu, China

CHUNRONG FANG, Nanjing University, Nanjing, Jiangsu, China

DING YANG, Nanjing University, Nanjing, Jiangsu, China

SHUN LI, Nanjing University, Nanjing, Jiangsu, China

BINYU LI, Nanjing University, Nanjing, Jiangsu, China

[View all](#)

Open Access Support provided by:

[Nanjing University](#)

Published: 19 December 2024

Online AM: 26 March 2024

Accepted: 14 March 2024

Revised: 31 January 2024

Received: 26 July 2023

[Citation in BibTeX format](#)

DiPri: Distance-Based Seed Prioritization for Greybox Fuzzing

RUIXIANG QIAN, SKLNST, Nanjing University, Nanjing, China

QUANJUN ZHANG, SKLNST, Nanjing University, Nanjing, China

CHUNRONG FANG, SKLNST, Nanjing University, Nanjing, China

DING YANG, SKLNST, Nanjing University, Nanjing, China

SHUN LI, SKLNST, Nanjing University, Nanjing, China

BINYU LI, SKLNST, Nanjing University, Nanjing, China

ZHENYU CHEN, SKLNST, Nanjing University, Nanjing, China

Greybox fuzzing is a powerful testing technique. Given a set of initial seeds, greybox fuzzing continuously generates new test inputs to execute the program under test and drives executions with code coverage as feedback. Seed prioritization is an important step of greybox fuzzing that helps greybox fuzzing choose promising seeds for input generation in priority. However, mainstream greybox fuzzers like AFL++ and Zest tend to neglect the importance of seed prioritization. They may pick seeds plainly according to the sequential order of the seeds being queued or an order produced with a random-based approach, which may consequently degrade their performance in exploring code and exposing bugs. In the meantime, existing state-of-the-art techniques like Alphuzz and K-Scheduler adopt complex strategies to schedule seeds. Although powerful, such strategies also inevitably incur great overhead and will reduce the scalability of the proposed technique.

In this article, we propose a novel distance-based seed prioritization approach named DiPri to facilitate greybox fuzzing. Specifically, DiPri evaluates the queued seeds according to seed distances and chooses the outlier ones, which are the farthest from the others, in priority to improve the probabilities of discovering previously unexplored code regions. To make a profound evaluation of DiPri, we prototype DiPri on AFL++ and conduct large-scale experiments with four baselines and 24 C/C++ fuzz targets, where eight are from widely adopted real-world projects, eight are from the coverage-based benchmark FuzzBench, and eight are from the bug-based benchmark Magma. The results obtained through a fuzzing exceeding 50,000 CPU hours suggest that DiPri can (1) insignificantly influence the host fuzzer's capability of code coverage by slightly improving the branch coverage on the eight targets from real-world projects and slightly reducing the branch coverage on the eight targets from FuzzBench, and (2) improve the host fuzzer's capability of finding bugs by triggering five more Magma bugs. Besides the evaluation with the three C/C++ benchmarks, we integrate DiPri into the Java fuzzer Zest and conduct experiments on a Java benchmark composed of five real-world programs for more than 8,000 CPU hours to empirically study the scalability of DiPri. The results with the Java benchmark demonstrate that DiPri is pretty scalable and can help the host fuzzer find bugs more consistently.

This work was partially supported by the National Natural Science Foundation of China (61932012, 62372228, 62272220), and the Science, Technology, and Innovation Commission of Shenzhen Municipality (CJGJZD20200617103001003).

Authors' address: R. Qian, Q. Zhang, C. Fang (Corresponding author), D. Yang, S. Li, B. Li, and Z. Chen (Corresponding author), State Key Laboratory of Novel Software Technology (SKLNST), Nanjing University, Nanjing, Jiangsu, China; e-mails: qianrx@smail.nju.edu.cn, quanjun.zhang@smail.nju.edu.cn, fangchunrong@nju.edu.cn, 522023320179@smail.nju.edu.cn, 522023320091@smail.nju.edu.cn, 191250066@smail.nju.edu.cn, zychen@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1049-331X/2024/12-ART26

<https://doi.org/10.1145/3654440>

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: Greybox fuzzing, seed prioritization, seed distance

ACM Reference Format:

Ruixiang Qian, Quanjun Zhang, Chunrong Fang, Ding Yang, Shun Li, Binyu Li, and Zhenyu Chen. 2024. DiPRI: Distance-Based Seed Prioritization for Greybox Fuzzing. *ACM Trans. Softw. Eng. Methodol.* 34, 1, Article 26 (December 2024), 39 pages. <https://doi.org/10.1145/3654440>

1 INTRODUCTION

Fuzzing is a software testing technique famous for its deployment simplicity and power in exposing potential software bugs [45]. The core of fuzzing is a loop where the fuzzer (the tool that implements fuzzing) continuously generates test inputs from a given set of initial seeds and executes the **program under test (PUT)** [50]. Greybox fuzzing is an essential branch of fuzzing that leverages execution status collected via lightweight instrumentation (e.g., code coverage) as the feedback for guidance [51]. Compared to blackbox fuzzing, which executes the PUT somewhat blindly, and whitebox fuzzing, which may incur huge costs due to adopting heavy program analysis techniques, greybox fuzzing achieves a better balance in effectiveness and efficiency [82]. Such balance helps it discover crashes well and makes it a hot spot of recent research [13, 40, 72, 77].

Seeds, or seed inputs, are critical for greybox fuzzing, as they fundamentally shape the nature of the offspring inputs generated atop them and thereby determine the trend of fuzzing to a large extent [15, 78]. In the workflow of greybox fuzzing, seed prioritization is performed to identify and optimize promising seeds [66]. Specifically, the queued seeds are first evaluated by certain metrics and then reordered for subsequent input generation with respect to the results of the evaluation. Although seed prioritization can greatly affect the performance of greybox fuzzing by generating offspring inputs from the promising seeds earlier, its importance tends to be slighted by mainstream greybox fuzzers. For example, the most famous greybox fuzzer AFL [1] and the mainstream Java greybox fuzzer Zest [54] both adopt an order-of-queued prioritization approach, which passes seeds to input generation in the sequential order of how they are added to the queue. Besides the order-of-queued approach, AFL++ [2], the community-driven superior fork of AFL, employs a weighted random-based prioritization approach powered by alias tables. Both of the two preceding approaches are somewhat blind and can starve the seeds that are actually promising for new coverage, which may consequently restrict the performance of the host greybox fuzzer.

Besides seed prioritization approaches, there are other fields of fuzzing interested in determining the sequence of seeds [13, 81]. Seed scheduling is a good example that focuses on selecting seeds for mutation or input generation with novel strategies [34, 42]. For example, K-Scheduler employs graph centrality analysis to identify horizon edges and prioritizes the seeds that attain the highest centrality scores [66]. Alphuzz constructs the seed mutation trees with the mutation relationships among seeds and leverages the constructed trees to schedule seeds as a Monte Carlo Tree Search problem [80]. Although effective, these techniques are built upon complex ideas requiring sophisticated operations, which inevitably incur high computational costs and prevent them from scaling.

Inspired by the thought of **adaptive random testing (ART)** that chooses diverse tests according to the distances among tests [19, 36], we propose DiPRI, a novel yet simple distance-based seed prioritization approach to boost greybox fuzzing. Once integrated into a greybox fuzzer, DiPRI operates in two steps to rearrange seeds for input generation. First, DiPRI conducts a distance-based seed evaluation by (1) calculating seed distances incrementally with the coverage vectors achieved

by the seeds and (2) determining priority scores for each seed by accumulating the distances between it and the other seeds. Second, DiPRI conducts a seed reordering to rearrange the queued seeds with the calculated priority scores. Both the definition of coverage vector and the distance measures are customizable for testers. In addition, we design three prioritization modes, namely VANILLA, PERIODICAL, and ADAPTIVE, to help testers adjust the frequency of activating DiPRI.

To evaluate DiPRI and investigate the characteristics of its configurations and components, we integrate DiPRI into AFL++ and conduct well-designed experiments with four baselines (i.e., two seed prioritization approaches and two **state-of-the-art (SOTA)** seed scheduling techniques), two distance measures (i.e., Euclidean and Hamming), and 24 fuzz targets from three widely adopted C/C++ benchmarks. Among the 24 evaluated targets, 8 of which are from real-world projects [39, 79], 8 are from the coverage-based benchmark FuzzBench [14, 49], and the remaining 8 are from the bug-based benchmark Magma [28, 30]. The experimental results produced by fuzzing for exceeding for 50,000 hours reveal the following. First, as for different DiPRI configurations, the ADAPTIVE mode is the most effective among the three proposed prioritization modes, whereas Hamming performs better than Euclidean as the distance measure configured in DiPRI (Section 5.1). Second, comparing with the two baseline seed prioritization approaches (i.e., the order-of-queued approach and the random-based approach), DiPRI performs no significant strength in terms of code coverage but helps AFL++ trigger five more Magma bugs, indicating that DiPRI can improve the bug-finding capability of the host fuzzer (Section 5.2). Third, comparing with the two baseline seed scheduling techniques (i.e., K-Scheduler and Alphuzz), DiPRI performs the best when configured with the ADAPTIVE mode and Hamming measure, which demonstrates its competitiveness to the SOTA (Section 5.3). Fourth, when conducting seed reordering with the priority scores calculated by other metrics widely adopted in other fuzzing practices, the performance of DiPRI is degraded in terms of code coverage, which suggests the superiority of the proposed distance-based metric (Section 5.4). Besides evaluating DiPRI on C/C++ benchmarks, we integrate DiPRI into the mainstream Java fuzzer Zest and conduct a fuzzing for 8,050 hours on five Java fuzz targets to empirically investigate how DiPRI scales to another scenario. The empirical results show the following. First, DiPRI is pretty scalable and can be integrated easily into another host fuzzer implemented with a different programming language. Second, although it may slightly reduce the code coverage, DiPRI can still help the host fuzzer in finding bugs by improving the hit counts as well as the consistency of the triggered bugs. In summary, the contributions of this article are as follows:

- *Novel approach*: To the best of our knowledge, DiPRI is the first distance-based seed prioritization that prioritizes the outlier seeds. The framework of DiPRI is both innovative and general, which can be easily scaled to new fuzz targets and new testing scenarios.
- *Practical implementation*: We prototype DiPRI in both C/C++ and Java by integrating it into AFL++ and Zest. To support open science and provide assistance to follow-up researchers, we release the source code and other artifacts of DiPRI on a public website [21].
- *Profound study*: To the best of our knowledge, we conduct the first cross-language empirical study involving two programming languages (i.e., C/C++ and Java), two mainstream host fuzzers (i.e., AFL++ and Zest), four baselines, and 29 fuzz targets from four benchmarks (i.e., three C/C++ benchmarks and one Java benchmark). The empirical results extracted from a fuzzing exceeding 7 CPU years can provide not only evidence for the evaluation of DiPRI but also insights for follow-up research.

2 BACKGROUND AND MOTIVATION

In this section, we first introduce the general frameworks of greybox fuzzing (Section 2.1) and ART (Section 2.2), then show the motivation of DiPRI with an illustrative example (Section 2.3).

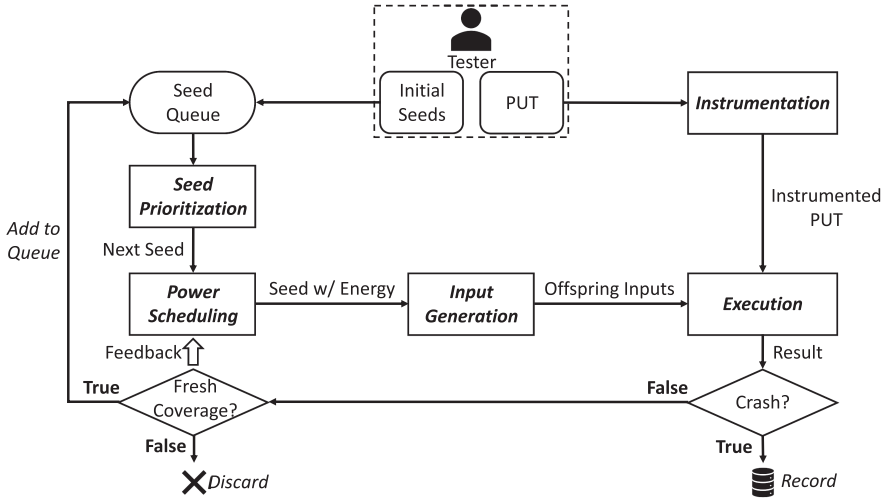


Fig. 1. General workflow of greybox fuzzing.

2.1 Greybox Fuzzing

Greybox fuzzing is an essential family of fuzzing [45, 82]. Figure 1 illustrates the general workflow of greybox fuzzing. To make a preparation, greybox fuzzing first constructs the seed queue with the initial seeds provided by the tester and instruments the PUT with coverage-tracing instructions to get ready for runtime coverage collection. It then operates in a loop, namely the main fuzzing loop, to continuously generate test inputs to execute the PUT. The main fuzzing loop can be partitioned into four steps: (1) *seed prioritization*, (2) *power scheduling*, (3) *input generation*, and (4) *execution*. At each iteration of the loop, greybox fuzzing first prioritizes the seed queue with respect to certain criteria and chooses the most promising seed for input generation. It then conducts power scheduling to assign energy to the chosen seed to decide the time budget that the seed should act as the parent of generation [13]. With the parent seed and the assigned energy, greybox fuzzing continuously generates offspring test inputs by imposing various mutators upon the seed until the energy runs out [24, 68]. Every offspring input is executed against the PUT to explore code regions and probe potential crashes. According to the execution result, an offspring input can be (1) recorded if it has triggered any crash, or (2) added to the queue if it has uncovered fresh coverage, or (3) discarded if it has no contribution to bug exposure and code coverage.

Seed prioritization is a critical step of greybox fuzzing, as it can influence the trends of fuzzing by dictating the next seed exploited for generating offspring inputs. However, mainstream greybox fuzzers, such as AFL++ [2] and Zest [54], tend to slight the importance of seed prioritization. In this article, we propose a distance-based seed prioritization approach, namely DiPRI, to facilitate greybox fuzzing by prioritizing seeds that are the most distant from others. We illustrate how DiPRI works with an example in Section 2.3 and elaborate on its technical designs in Section 3.

2.2 Adaptive Random Testing

ART was first proposed by Chen et al. [19] to facilitate random testing. The essential insight behind ART is to select diverse tests from different code regions as failures tend to cluster into contiguous regions [19, 36, 60]. With various intuitions, ART can be implemented in many different ways [36]. **Fixed-Size-Candidate-Set (FSCS)** is one of the most widely adopted implementations of ART, which picks a fixed-size of tests at each iteration of selection [19, 36]. Suppose C is the set of

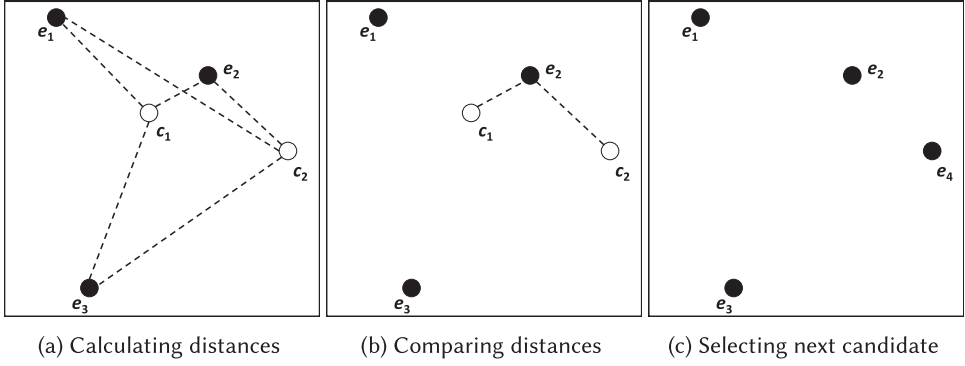


Fig. 2. The exemplified process of FSCS complying the constraint τ .

candidate tests for selection and E is the set of tests that have already been selected and executed. The goal of FSCS is to pick k ($k \in \mathbb{N}^*$) tests that are farthest away from E with respect to a certain constraint. For example, a common constraint is to select a test that has the maximum minimum distance to all of the executed tests, which can be denoted as follows:

$$\tau : \forall c \in C, \min_{e \in E} \mathcal{D}(c', e) \geq \min_{e \in E} \mathcal{D}(c, e), \quad (1)$$

where τ is the notation of the constraint, $\mathcal{D}$ is the distance function for calculating the distance between two tests, c and e are the elements of C and E , and c' is the candidate test selected in an iteration of FSCS satisfying τ . Figure 2 illustrates the process of FSCS complying the constraint τ , where the executed test set is $E = \{e_1, e_2, e_3\}$ and the candidate set is $C = \{c_1, c_2\}$. To select a test from C for the next execution, FSCS first calculates the distance $\mathcal{D}(e, c)$ between each pair of candidate and executed tests (see Figure 2(a)). The distances between tests are represented with dashed lines. With the calculated distances, FSCS identifies the candidate test that is the farthest from the executed ones with considered constraint τ (see Figure 2(b)). The candidate test c_2 is selected as the next test e_4 for execution, as it has a larger minimum distance than c_1 (see Figure 2(c)).

Inspired by the thought of ART, we propose DiPRI to prioritize queued seeds according to seed distances during greybox fuzzing. Our insight is that a seed that is the most distant from others processes the highest probability of uncovering unexplored code regions. We concretize such insight with an example in Section 2.3 and elaborate on the distance-based seed prioritization in Section 3.2.

2.3 A Motivating Example

This section presents the targeted problem scope of this article and shows how DiPRI works through an illustrative example. Given a queue of seeds S , the goal of DiPRI is to prioritize and choose the *outlier seed* from S for the next round of input generation. The *outlier seed* is defined as the seed that is the farthest from the others in the seed queue with respect to the constraint given by the tester. Figure 3 presents a scenario that DiPRI may face during fuzzing, where the input domain is simplified to a 2D space. The seed queue at the current moment is denoted as $S = \{s_1, s_2, s_3, s_4, s_5\}$, where the subscripts $\{1, 2, 3, 4, 5\}$ represent the order in which the seeds are added to the queue. The solid dots in Figure 3(b) stand for the seeds and the light-colored circles simulate the regions that can be explored by the offspring inputs generated from the seeds located at the centers. For ease of presentation, we ignore the possibility that the selected seeds can give birth to test inputs that may be able to explore very distant regions [76]. Recall the two

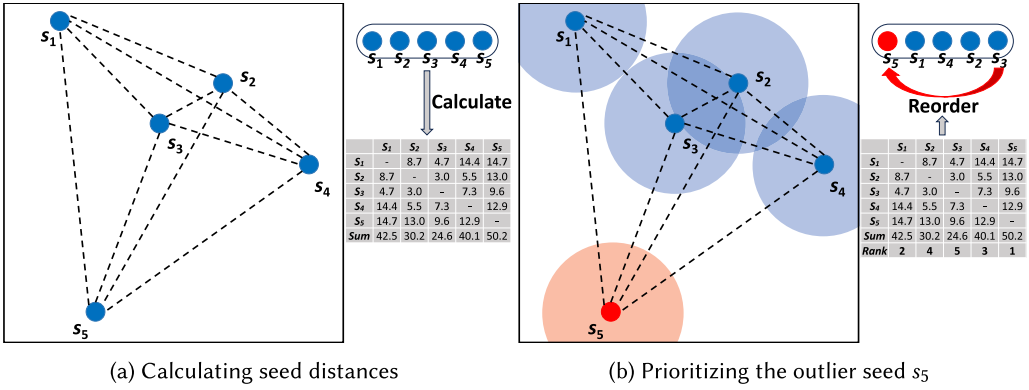


Fig. 3. An illustration of identifying and prioritizing the outlier seed.

prioritization approaches adopted by the mainstream fuzzers (i.e., AFL, AFL++, and Zest) described in Section 1, both of which are unaware of the distribution of seeds. Specifically, the order-of-queued approach trivially picks a seed in the order of s_1 to s_5 and thus can drive the fuzzers to explore overlapped regions repeatedly. The random-based approach is more likely to pick seeds from $\{s_1, s_2, s_3, s_4\}$ rather than choosing s_5 . As a result, it may also steer fuzzers to waste resources in densely explored regions. Unlike these two approaches, DiPRI chooses to calculate the distances among seeds to identify their distributions first (see Figure 3(a)). It then evaluates each seed with respect to the calculated distances and prioritizes the outlier one s_5 for the next round of input generation (see Figure 3(b)). With the help of DiPRI, greybox fuzzing can avoid densely explored regions and allocate resources for rarely explored ones in priority.

3 APPROACH

In this section, we elaborate on the formulation and technical details of DiPRI. We start by presenting the high-level workflow of DiPRI (Section 3.1) and then delve into the technical details (Sections 3.2–3.3).

3.1 Overview of DiPRI

DiPRI is essentially a plugin for greybox fuzzing. It relies on two additional steps to take effects: a *distance-based seed prioritization* and a *coverage-centered result analysis*. The workflow of fuzzing with DiPRI is shown as Figure 4. Specifically, distance-based seed prioritization is the core of DiPRI, which identifies and picks the outlier seed for the next round of input generation. The distance-based seed prioritization can be subdivided into two stages, namely seed evaluation and seed reordering. At the seed evaluation stage, DiPRI evaluates the potentials of seeds by calculating seed distances with respect to a given constraint. We refer to such potentials as priority scores and use them as the credentials for the seed reordering stage. The coverage-centered result analysis is a run-up of the distance-based seed prioritization. It parses coverage vectors (i.e., the inputs of distance calculation) from the execution results and binds them to the executed test inputs. Note that the coverage vectors can vary when DiPRI is configured with different distance measures.

Algorithm 1 further exhibits the pseudo-code of the greybox fuzzing equipped with DiPRI, where the additional steps of DiPRI are highlighted in orange. After initializing the two seed queues and coming into the main fuzzing loop, DiPRI first evaluates every seed queued in S' with respect to the distance measure given with the configuration *conf* (line 3). It then reorders the queued seeds by the calculated priority scores (line 4). These two lines make up the

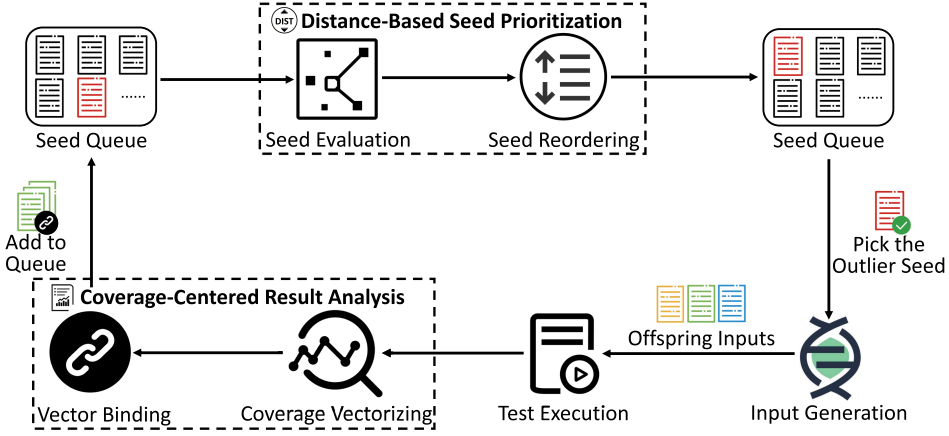


Fig. 4. The overview of fuzzing with DIPRI.

aforementioned two stages of distance-based seed prioritization, and their details are further presented in Sections 3.2 and 3.3. After reordering, the outlier seed is picked from the top of the queue S' and assigned energy to generate offspring inputs (line 7). Every generated input is executed against the instrumented PUT, and the execution result is collected (line 8). Based on the execution result, DIPRI parses the coverage vector with respect to the distance measure given with *conf* (line 9) and binds it to the corresponding input s' (line 10). After vectorizing coverage, the underlying greybox fuzzer continues to arrange the offspring inputs according to their execution results. The inputs that lead to crashes are added to the crashing seed queue S^* for later reproduction (line 12), and the interesting seeds that attain fresh coverage are added to S' to support subsequent fuzzing (line 14).

3.2 Distance-Based Seed Evaluation

Seed distances can be quantified by different distance measures from many perspectives. For example, the Euclidean distance measure can be leveraged to quantify the distances among the contents of numeric test inputs [36, 37], whereas the Hamming and Jaccard distance measures can be exploited to quantify the coverage distances among test cases [52, 58, 69]. In this article, we focus on execution coverage and leverage coverage distances as seed distances. Given a seed s , the coverage vector v of s is a list of coverage numbers that can be denoted as

$$v(s) = \{x^1, x^2, \dots, x^n\}, x^i \in \Sigma, i \in \{1, 2, \dots, m\}, m \in \mathbb{N}^*, \quad (2)$$

where m is the length of the v , $\mathbb{N}^*$ is the set of positive integers, and Σ is the alphabet of values possible for the coverage numbers. Note that the member values of Σ are decided together by the considered distance measure and the implementation of host fuzzers. For example, when the distance measure is Euclidean and the fuzzer is AFL++, the elements of Σ are $\{0, 1, \dots, 255\}$ as AFL++ records edge coverage with eight-bit unsigned integers [2]. In contrast, when considering the Hamming distance measure and keeping AFL++ as the host fuzzer, the elements of Σ are $\{0, 1\}$, as Hamming requires binary-valued inputs [27]. Suppose S is a seed queue consisting of n ($n \in \mathbb{N}^*$) seeds and s_i, s_j ($i, j \in \{1, 2, \dots, n\}$) are two seeds from S . Let v_i, v_j be the coverage vectors of s_i, s_j and $\mathcal{D}$ be the distance function constructed with respect to the considered measure. The seed distance between s_i and s_j is defined as the distance between their vectors, which establishes

$$\mathcal{D}(s_i, s_j) = \mathcal{D}(v_i, v_j). \quad (3)$$

ALGORITHM 1: Greybox Fuzzing with DiPri**Input:** Initial Seeds S , Instrumented PUT p , DiPri Configuration $conf$ **Output:** Seed Queue S' , Crashing Seed Queue S^*

```

1:  $S' \leftarrow S, S^* \leftarrow \emptyset$ 
2: repeat ▷ Main fuzz loop
3:    $S' \leftarrow \text{EVALUATEBYDISTANCE}(S', conf)$ 
4:    $s \leftarrow \text{REORDERANDPICK}(S', conf)$ 
5:    $e \leftarrow \text{ASSIGNENERGY}(s)$ 
6:   for  $i$  from 1 to  $e$  do
7:      $s' \leftarrow \text{GENERATENEWINPUT}(s)$ 
8:      $res \leftarrow \text{EXECUTE}(p, s')$ 
9:      $v \leftarrow \text{PARSEDISTVEC}(res, conf)$ 
10:     $\text{BINDINPUTWITHVEC}(s', v)$ 
11:    if  $\text{ISCRASH}(s', res)$  then
12:      add  $s'$  to  $S^*$ 
13:    else if  $\text{ISINTERESTING}(s', res)$  then
14:      add  $s'$  to  $S'$ 
15:    end if
16:  end for
17: until resources exceeds or abort-signal comes
18: return  $S', S^*$ 

```

To identify and prioritize the outlier seed that is the farthest from the others, we define the priority score of a seed s_i as the sum value of the distances between s_i and other seeds within S . With Equation (3), the calculation for the priority score $\mathcal{P}$ of s_i can be formalized as

$$\mathcal{P}(s_i) = \sum_{j=1}^n \mathcal{D}(s_i, s_j) = \sum_{i=1}^n \mathcal{D}(v_i, v_j). \quad (4)$$

Note that the concrete calculations of the seed distances vary for the used measures. We consider two distance measures in our evaluation to investigate how distance measures influence the performance of DiPri. The calculations with respect to distance measures are described in Section 4.2.1.

3.3 Seed Reordering according to Priority Scores

As introduced in Section 2.2, ART chooses the most distant test cases for the next execution according to a preset constraint (Equation (1)). The insight of such design is to improve test diversity via visiting the remote code regions as early as possible [19]. Inspired by the idea of ART, we propose to pick the outlier seed in priority for each round of input generation. We achieve this goal by letting DiPri reorder the queued seeds with respect to the priority score calculated by Equation (4), which are essentially the sums of seed distances. With the notations introduced in Section 3.2, the constraint τ' that DiPri complies for seed reordering can be formalized as

$$\tau' : \forall s_i, s_j \in S, \mathcal{P}(s_i) \geq \mathcal{P}(s_j) \implies s_i \geq s_j, \quad (5)$$

where $s_i \geq s_j$ represents the partial order that the seed s_i is arranged in front of or equal to another seed s_j . Equation (5) implies that seeds with larger priority scores are ranked higher within the queue. Note that Equation (5) is general and can work with priority scores besides the one defined by Equation (4). For instance, one can define the priority score as the reciprocal of input size to rank smaller seeds higher through seed reordering. In our evaluation, we design experiments using

ALGORITHM 2: Incremental Distance-Based Seed Prioritization**Input:** Seed Queue S , DiPRI Config $conf$ **Output:** Prioritized Seed Queue S_p

```

1:  $S_p \leftarrow S$ 
2:  $n \leftarrow \text{GETQUEUELEN}(S)$ 
3: if HASGROWN( $S$ ) and REACHTIMMING( $conf$ ) then
4:   for  $i$  from 1 to  $n$  do                                      $\triangleright$  Update priority scores.
5:      $s_i \leftarrow \text{GETSEED}(S_p, i)$ 
6:     if ISFRESH( $s_i$ ) then                                      $\triangleright$  Focus on freshly added seeds.
7:       for  $j$  from 1 to  $i$  do
8:          $s_j \leftarrow \text{GETSEED}(S_p, j)$ 
9:          $d \leftarrow \text{CALDIST}(s_i, s_j, conf)$ 
10:        UPDATEPRI SCORE( $s_i, d$ )
11:        UPDATEPRI SCORE( $s_j, d$ )
12:      end for
13:    end if
14:  end for
15:   $S_p \leftarrow \text{REORDERBYPRI SCORES}(S_p, conf)$               $\triangleright$  Reorder seeds.
16: end if
17: return  $S_p$ 

```

different priority scores calculated with other metrics to investigate how effective the proposed distance-based priority score calculation is. The experimental results are discussed in Section 5.3.

3.4 Incremental Distance-Based Seed Prioritization

Although ART improves the diversity of testing, it also suffers from the heavy overhead brought by distance calculation [46]. DiPRI adopts an incremental calculation to alleviate such overhead. Algorithm 2 illustrates the pseudo-code of the incremental seed prioritization, which is a composition of the distance-based seed evaluation and the seed reordering. DiPRI first checks whether the condition of prioritizing has been satisfied according to two factors: (1) whether the queue S has grown in previous fuzzing and (2) whether the timing is reached with respect to the prioritization mode configured in $conf$ (line 3). In this article, we design three prioritization modes, namely VANILLA, PERIODICAL, and ADAPTIVE. The timing for prioritizing varies for each of the three modes. Specifically, VANILLA prioritizes every time the seed queue has grown; PERIODICAL prioritizes when the time elapsed since the last prioritization has exceeded a preset period δ ; and ADAPTIVE prioritizes when all last prioritized seeds have already been exhausted for input generation. Note that to refrain PERIODICAL from degrading to the plain order-of-queued approach, we impel it to prioritize if all of the last prioritized seeds are exhausted before the preset period δ is reached, which means that PERIODICAL can convert to ADAPTIVE if δ is configured too long. When the prioritization condition is satisfied, DiPRI performs the distance-based seed evaluation to update the priority scores for the queued seeds (lines 4–14). To reduce the computation overhead, we merely focus on seeds newly added since the last prioritization (line 6) and increase their maintained priority scores with the incrementally calculated seed distances (lines 10 and 11). After updating the priority scores of seeds, DiPRI reorders S_p complying with the constraint τ' defined by Equation (5) (line 15). The prioritized seed queue S_p is finally returned to assist the next round of input generation (line 17).

4 EVALUATION

This section presents the methodology for evaluating DiPRI. We start by describing the focused research questions (Section 4.1) and then elaborate on the experimental setups (Section 4.2).

4.1 Research Questions

We concentrate on the following research questions in the evaluation:

- RQ1: How does DiPRI perform under different configurations?
- RQ2: How does DiPRI compare to other seed prioritization approaches?
- RQ3: How does DiPRI compare to SOTA seed scheduling techniques?
- RQ4: How effective is the distance-based seed evaluation?

Specifically, RQ1 aims at analyzing the characteristics of different DiPRI configurations, which are combinations of two distance measures and three prioritization modes. RQ2 compares DiPRI with the two seed prioritization approaches (i.e., the order-of-queued and random-based approaches) built in the host fuzzer AFL++. RQ3 explores whether DiPRI is comparable to SOTA seed scheduling techniques, such as Alphuzz [80] and K-Scheduler [66]. RQ4 investigates how the two components of DiPRI (i.e., the distance-based seed evaluation and the seed reordering) benefit the host fuzzer. The experimental results of RQ1 through RQ4 respectively are presented in Sections 5.1 through 5.4.

4.2 Experimental Setups

The experimental setups consist of five parts, namely the two considered distance measures, the prototype implementation of DiPRI atop AFL++, the fuzz targets, the effectiveness indicators, and the information about the fuzz campaigns as well as the experiment infrastructures.

4.2.1 Distance Measures. We consider the Hamming and Euclidean distance measures in the evaluation. The reasons we choose these two measures are twofold. First, both of the measures are widely adopted in prior research on software testing and other fields [9, 20, 27, 52]. Second, the two measures have disparate definitions and thus can profile coverage distances differently. The calculations of seed distances and the alphabets concerning the two measures are shown as follows:

$$\mathcal{D}_H(s_a, s_b) = \mathcal{D}_H(v_a, v_b) = \sum_{i=1}^k x_a^i \oplus x_b^i, \Sigma_H = \{0, 1\}, \quad (6)$$

$$\mathcal{D}_E(s_a, s_b) = \mathcal{D}_E(v_a, v_b) = \sqrt{\sum_{i=1}^k (x_a^i - x_b^i)^2}, \Sigma_E = \{0, 1, 2, \dots, 255\}. \quad (7)$$

In the two preceding equations, s_a and s_b are two seeds and v_a and v_b are their coverage vectors. The $x_{(\cdot)}^i$ denotes the i th coverage number of the vector $v_{(\cdot)}$. The $\Sigma_{(\cdot)}$ represents the alphabet of a certain distance measure, where we use H and E to denote the Hamming and Euclidean distance measures, respectively. We do not include the Jaccard distance measure in the current evaluation, as Jaccard and Hamming are similar measures requiring binary-valued inputs [58]. Besides, we have observed similar performances of DiPRI with Hamming and Jaccard in our preliminary evaluation. The results of the preliminary evaluation can be found in the FUZZING'23 version of this work [56].

4.2.2 DiPRI Implementation. We prototype DiPRI by extending AFL++ (version 4.06c) [2]. Specifically, we construct new data structures to put the variables needed by DiPRI and expand the data structure that AFL++ uses for representing seeds to record coverage vectors as well as priority

Table 1. Information of the Experimental Fuzz Targets

	TID	Project & Version	Fuzz Target	Initial Seeds
Real World	T01	binutils-2.40	cxxfilt	"_Z1fv"
	T02		nm-new	afl-tc/small_exec.elf
	T03		objdump -d	
	T04		readelf -a	
	T05	libjpeg-turbo-2.1.5.1	djpeg	afl-tc/not_kitty.jpg
	T06	mjs-2.20.0	mjs -f	afl-tc/small_script.js
	T07	mupdf-1.21.1	mutool draw	afl-tc/small.pdf
	T08	libxml2-2.10.3	xmllint	afl-tc/small_document.xml
FuzzBench	T09	bloaty_fuzz_target	bloaty_fuzz_target	FuzzBench-Default [26]
	T10	curl_curl_fuzzer_http	curl_curl_fuzzer_http	
	T11	freetype2-2017	freetype2_ftfuzzer	
	T12	jsoncpp_jsoncpp_fuzzer	jsoncpp_jsoncpp_fuzzer	
	T13	lcms-2017-03-21	lcms_cms_transform_fuzzer	
	T14	libpcap_fuzz_both	libpcap_fuzz_both	
	T15	re2-2014-12-09	re2_fuzzer	
	T16	sqlite3_ossfuzz	sqlite3_ossfuzz	
Magma	T17	libpng	libpng_read_fuzzer	Magma-Default [44]
	T18	libsndfile	sndfile_fuzzer	
	T19	libtiff	tiff_read_rgba_fuzzer	
	T20		tiffcp	
	T21	libxml2	xmllint	
	T22		xml_read_memory_fuzzer	
	T23	lua	lua	
	T24	sqlite3	sqlite3_fuzz	

scores. Moreover, we revise AFL++ source files (e.g., `afl-fuzz-queue.c` and `afl-fuzz-bitmap.c`) to support DiPRI’s functionalities, such as different flavors of seed evaluations and quick-sort-based seed reordering. We publicize the source code and artifacts of DiPRI to support open science [21].

4.2.3 Fuzz Targets. We conduct experiments on 24 fuzz targets crossing 18 file types, of which 8 are from real-world projects adopted by prior works [39, 65, 67], 8 are from the widely recommended benchmark project FuzzBench [49], and the rest are from the artificial buggy benchmark Magma [28]. The details of the fuzz targets are shown at Table 1, where the five columns respectively exhibit the benchmark, the IDs of the targets, the project and the version (if any), the fuzz target, and initial seeds prepared for fuzz campaigns. For the targets from real-world projects, we build them by downloading the source code of the version newest at the time the experiments are prepared and instrumenting them with `afl-cc`, the coverage instrumentor provided by AFL++, under the default setting. For the targets from FuzzBench and Magma, we build them following the standard instructions provided by the benchmark developers. As for the initial seeds, we follow Lemieux and Sen [39] to prepare “_Z1fv” for `cxxfilt` and pick one file from the AFL test cases [4] (denoted as “afl-tc”) for each of the remaining targets, whereas for targets from FuzzBench and Magma, we use their default initial seed corpus. Note that for targets from real-world projects, we exhibit not only the name of the target program but also the specified command line option, as fuzzing a target program with different options may produce different results. Also note that although the Magma developers refer to the fuzzed projects as “targets” and the fuzzed programs as “drivers” [44], we do not follow them to keep unified terminologies for the three different benchmarks [26].

4.2.4 Effectiveness Indicators. We evaluate the performance of DiPRI and baselines with code coverage and bug-exposing capability following pioneer fuzzing research [14, 31, 39, 81]. We leverage branch coverage as the code coverage indicator, which is the default setting of FuzzBench [49]. For targets from real-world projects, we collect branch coverage by rerunning the test inputs generated and stored by the fuzzers with `llvm-cov` [41], which is a steady coverage tool widely adopted by prior works [30, 48]. As for bug-exposing capability, we adopt the setting of the Magma benchmark [28] and leverage the number of bugs and the time to reach/trigger bugs as indicators.

4.2.5 Campaigns and Infrastructures. A fuzz campaign is a specific execution of a fuzzer on a specific fuzz target lasting for a period of time [45]. We set the duration of fuzz campaigns to 23 hours following the recommendations of Böhme et al. [14]. For each of the fuzz campaigns, we repeat it for 10 trials to alleviate the impact of randomness. For fuzz campaigns on the eight targets from real-world projects, we conduct them on a machine with 16 cores (Intel Xeon Silver 4114 CPU @ 2.20 GHz) and 32 GB of memory running on the Ubuntu 22.04 operating system. For fuzz campaigns on the remaining targets from FuzzBench and Magma, we conduct them on a workstation with 32 cores (13th Gen Intel Core i9-13900K) and 32 GB of memory running on the Ubuntu 22.04 operating system.

5 RESULTS AND ANALYSES

In this section, we present the experimental results and conduct analyses upon them to answer the research questions raised in Section 4.1. We start by comparing and choosing the most powerful DiPRI configuration (RQ1 in Section 5.1) and then select configurations to compare to the seed prioritization approaches built in the host fuzzers to see whether can DiPRI make any improvements (RQ2 in Section 5.2). After that, we compare DiPRI to pioneer seed prioritization/scheduling techniques to see whether it is competitive (RQ3 in Section 5.3). Finally, we analyze the results of the ablation experiments to independently evaluate the contributions of DiPRI's two subcomponents (RQ4 in Section 5.4).

5.1 RQ1: Characteristics of Different DiPRI Configurations

5.1.1 DiPRI Configurations. We consider two distance measures (i.e., Hamming and Euclidean) and three prioritization modes (i.e., VANILLA, ADAPTIVE, and PERIODICAL) in the evaluation, whose modes make six configurations in total. Note that the period δ for PERIODICAL is preset to 1 minute in our experiments. To mark DiPRI configurations distinctively, we name them by connecting the prefix `dipri` with the initials of modes (V, A, or P) and distance measures (H or E), which are `dipri-VH`, `dipri-VE`, `dipri-AH`, `dipri-AE`, `dipri-PH`, and `dipri-PE` in particular.

5.1.2 Performance on Code Coverage. Figure 5 displays the tendencies of branch coverage that the six DiPRI configurations achieve over 10 trials of the 23-hour campaigns on the eight fuzz targets from real-world projects. The solid lines represent average branch coverage, and the light-colored regions depict ranges of branch coverage. Table 2 exhibits the average branch coverage that the six DiPRI configurations achieve on the eight targets from real-world projects, where the smaller-font values tagged at the right are the coverage-based ranks. The last row of Table 2 presents the average values of the coverage-based rank, and the smaller-font values are relative ranks. The first-ranked results are colored in green, and the second-ranked results are colored in yellow. With Figure 5 and Table 2, we have following findings. First, `dipri-AE` and `dipri-AH` together beat the other six configurations in seven of the eight targets, which indicates that ADAPTIVE mode is superior to PERIODICAL and VANILLA modes. Second, the Hamming-based configurations outperform the Euclidean-based configurations under each of the modes, which reveals that the

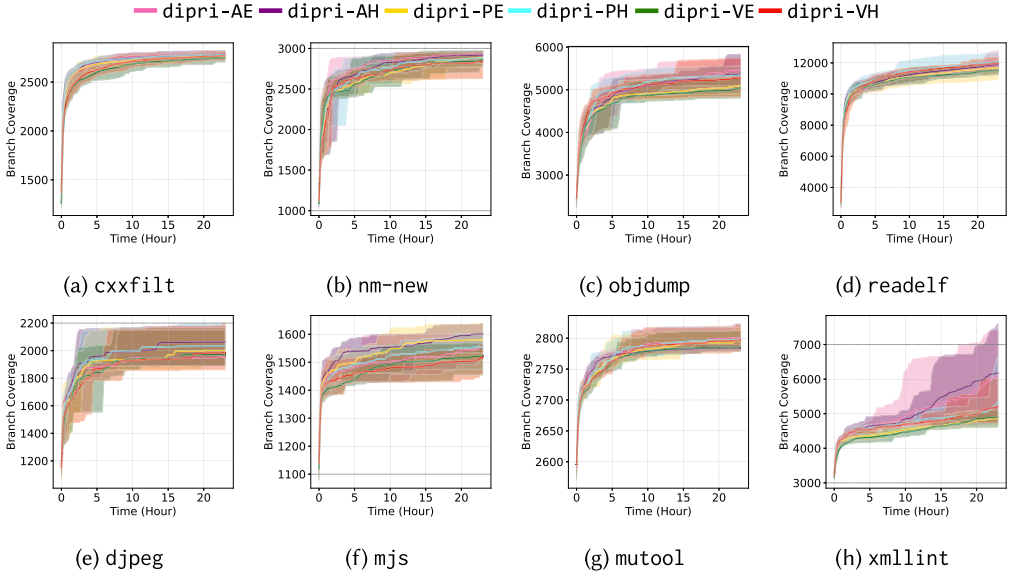


Fig. 5. The branch coverage that the six DiPRI configurations achieve over the 10 trials of the 23-hour campaign. The X axis is the elapsed time, whereas the Y axis is the branch coverage. The solid lines indicate the mean coverage, and the light-colored areas are the ranges between the maximum and minimum coverage.

Table 2. Average Branch Coverage (i.e., the Larger-Font Values on the Left of “/”) and the Coverage-Based Ranks (i.e., the Smaller-Font Values on the Right of “/”) That the Six DiPRI Configurations Achieve on the Eight Targets from Real-World Projects over 10 Trials of the 23-Hour Campaigns

Target	dipri-AE	dipri-AH	dipri-PE	dipri-PH	dipri-VE	dipri-VH
cxxfilt	2786.8 ₁	2782.6 ₂	2767.4 ₅	2780.3 ₃	2749.4 ₆	2771.1 ₄
nm-new	2931.9 ₁	2917.8 ₂	2880.3 ₄	2888.0 ₃	2860.8 ₅	2836.6 ₆
objdump	5461.3 ₁	5368.2 ₂	5096.0 ₅	5345.7 ₃	5041.1 ₆	5273.4 ₄
readelf	11808.9 ₄	11871.5 ₃	11683.3 ₅	12090.1 ₁	11523.1 ₆	11971.6 ₂
djpeg	1955.6 ₆	2061.7 ₁	2000.9 ₃	2032.1 ₂	1971.4 ₅	1985.0 ₄
mjs	1555.4 ₃	1601.9 ₁	1579.8 ₂	1554.8 ₄	1521.3 ₅	1517.0 ₆
mutool	2800.4 ₁	2784.1 ₆	2796.1 ₄	2797.3 ₂	2785.1 ₅	2796.3 ₃
xmllint	6089.5 ₂	6182.3 ₁	4863.2 ₆	5347.9 ₃	4897.6 ₅	5203.8 ₄
μ Rank	2.4 ₂	2.2 ₁	4.2 ₅	2.6 ₃	5.4 ₆	4.1 ₄

The “ μ Rank” row displays the average values of the coverage-based rank on the eight targets, where the smaller-font values are the relative ranks. The results ranked first are colored in green, and the results ranked second are colored in yellow.

Hamming measure is more suitable for DiPRI. Third, dipri-AH ranked first among all six configurations, whereas the second-ranked dipri-AE is closely comparable to it; the difference between the two ADAPTIVE configurations is much smaller than that between the two PERIODICAL configurations or the two VANILLA configurations, which implies that ADAPTIVE mode can weaken the impact of the distance measures. Figure 6 shows the distribution of the final code coverage achieved by each of DiPRI configurations over 10 trials of the 23-hour campaigns, where the median values are indicated with short bold lines. According to Figure 6, we find that dipri-AH

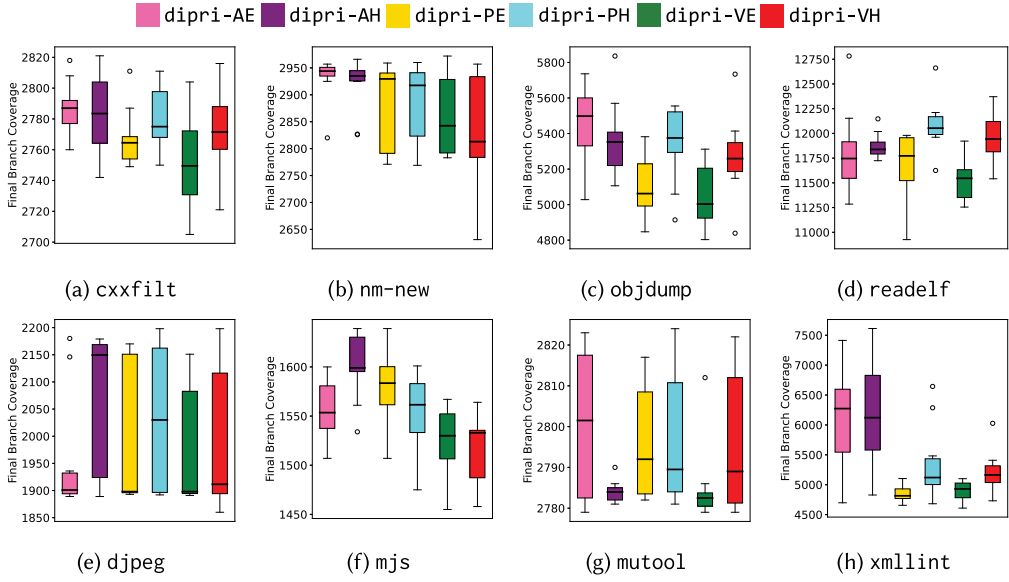


Fig. 6. The distributions of the final branch coverage that the six DiPRI configurations achieve at the end of each trial of the 23-hour campaign. The Y axis is the final branch coverage, and each box visualizes the results of a configuration. The short bold lines within the boxes denote median final branch coverage.

outperforms dipri-AE in terms of the average branch coverage but loses to dipri-AE on five out of eight targets in terms of median branch coverage. Besides, dipri-AH achieves the largest final branch coverage on five of the eight targets, which indicates a better capability of maximizing code coverage.

5.1.3 Overhead of DiPRI under Different Configurations. We record the time occupied by DiPRI to profile its overhead during fuzzing. Note that we do not distinguish the overheads brought by the distance-based seed evaluation and the seed reordering as the time consumed by seed reordering, which is usually less than a thousandth of the total, is insignificant compared to the time consumed by distance-based seed evaluation. Figure 7 depicts the tendencies of the occupied time in the process of the 23-hour campaigns under different DiPRI configurations. The X axis is the elapsed time and the Y axis is the percentages of the occupied time. The average occupancy is represented with solid lines, and the occupancy ranges are indicated with light-colored regions. According to Figure 7, the time occupied by the six DiPRI configurations presents a decreasing tendency as the fuzz campaigns advance on all eight targets. Such a decreasing tendency conforms to the nature of fuzzing, which is prone to finding more interesting test inputs (e.g., the ones that unveil new coverage) at an earlier stage [14, 55]. As we described in Section 3.4, DiPRI conducts an incremental distance-based prioritization that is triggered when the seed queue has grown. Finding interesting test inputs more frequently means a more frequent activation of DiPRI, and more executions of DiPRI naturally incur more time overhead. Besides the decreasing tendency, we find two other interesting phenomena with Figure 7. First, although the general trend is decreasing, the occupied time may quickly increase in the first 5 hours before starting to decline. Second, the occupied time is close on different configurations while varied on different fuzz targets. The reasons for the first phenomenon are twofold: on the one hand, the capacity of the seed queue is small at the very beginning of fuzzing such that DiPRI does not need to spend too much time on seed

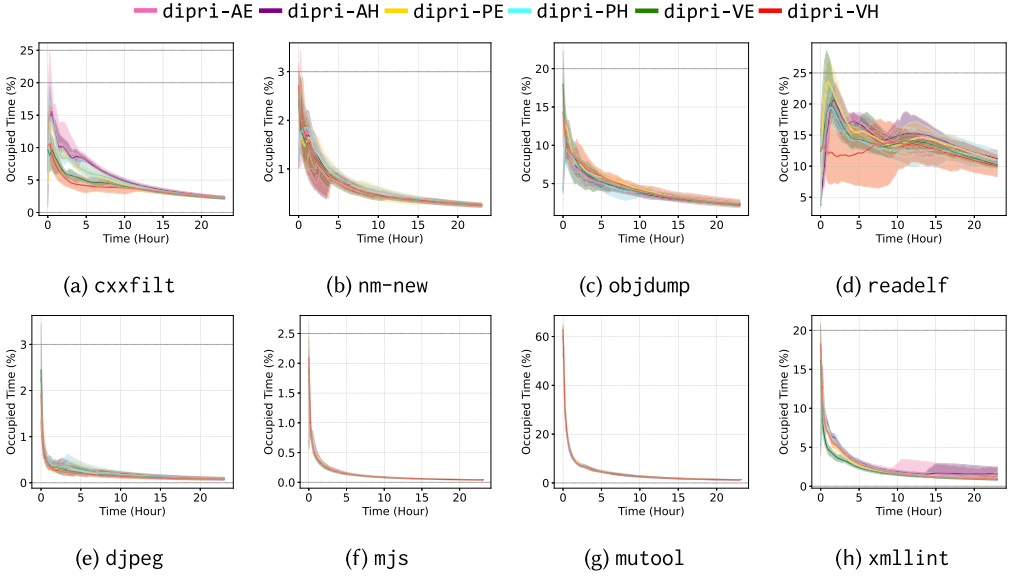


Fig. 7. The percentages of time occupied by DiPRI under the six configurations. The X axis is the elapsed time, and the Y axis is the percentage of occupied time. The solid lines are the average percentage of the time occupied by DiPRI, and light-colored regions visualize the ranges of time occupancy.

distance calculation, which consequently presents a low time occupancy; on the other hand, with many interesting test inputs found and the seed queue rapidly expanding, DiPRI is busy with calculating distances for a large amount of newly incoming seeds, leading to a sharp increase in time occupancy. As for the second phenomenon, we extract the real map size, the average final time occupancy, and the average number of DiPRI prioritization that is performed to make a further interpretation. The extracted data are presented in Table 3, where the “RMS” column displays the real map size of each target and the “ μ OTP” and “ μ NoP” respectively stand for average “Occupied Time Percentage” and “Number of Prioritizations.” Note that the RMS is the size of the bitmap that AFL++ leverages to record coverage information and is also the length of the coverage vectors used by DiPRI. The “ μ OTP” rows of Table 3 further indicate the close time occupancy of the six DiPRI configurations on all eight targets, which is surprising as we used to assume that the three modes of DiPRI may incur different time overhead. The close time occupancy of different configurations demonstrates two things. First, the proposed incremental distance-based prioritization is effective, as DiPRI does not incur more time overhead on any specific target when conducting more prioritizations under VANILLA or PERIODICAL mode (see “ μ NoP” rows in Table 3). Second, the performance differences among the six configurations are not associated with the extra overhead they bring. We later present that the performance differences of the six configurations are likely attributed to the different **seed-picking ratios (SPRs)** in Section 5.1.4. In contrast to the close time overheads on six configurations, the time overheads that DiPRI brings on the eight targets vary a lot, which range from 0.039% (dipri-PH on mjs) to 11.182% (dipri-AE on readelf) according to Table 3. The time overhead brought by DiPRI is co-determined by NoP and RMS, where NoP is the primary factor and RMS is the secondary factor. One concrete evidence that NoP is more significant than RMS to DiPRI’s time overhead is that, although readelf has a $0.2\times$ RMS of mutool, it spends DiPRI $7.9\times$ to $9.5\times$ time for prioritization as it activates DiPRI with $2.2\times$ to $5.9\times$ NoP.

Table 3. Experimental Results about the Overhead of DiPRI under the Six Configurations

Target	RMS		dipri-AE	dipri-AH	dipri-PE	dipri-PH	dipri-VE	dipri-VH
cxxfilt	37096	μ OTP	2.402	2.339	2.156	2.285	2.139	2.245
		μ NoP	331.800	316.400	926.700	930.700	3480.100	3637.100
nm-new	37614	μ OTP	0.264	0.253	0.263	0.271	0.266	0.262
		μ NoP	359.300	349.800	536.100	584.000	988.200	995.100
objdump	56381	μ OTP	2.353	2.280	2.385	2.347	2.211	2.340
		μ NoP	413.200	404.700	874.700	875.600	2125.300	2010.900
readelf	29116	μ OTP	11.182	11.144	10.945	10.906	10.209	10.019
		μ NoP	389.100	382.200	1128.100	1169.000	4692.400	3849.800
djpeg	11380	μ OTP	0.069	0.089	0.081	0.088	0.079	0.076
		μ NoP	221.200	214.400	471.100	487.400	828.500	809.800
mjs	4859	μ OTP	0.040	0.040	0.041	0.039	0.041	0.041
		μ NoP	379.600	391.400	512.600	518.500	1286.900	1245.400
mutool	167866	μ OTP	1.207	1.178	1.265	1.267	1.277	1.265
		μ NoP	173.600	173.500	419.100	417.100	791.500	824.500
xmllint	50802	μ OTP	1.457	1.579	0.866	1.053	0.831	0.962
		μ NoP	382.500	387.800	698.000	665.600	1571.800	1684.100

The “RMS” column displays the real map sizes of the target programs, which are also the lengths of the coverage vectors used by DiPRI during fuzzing. The “ μ OTP” rows show the average final occupied time percentages over 10 trials, and the “ μ NoP” rows show the average number of times that DiPRI prioritization is performed.

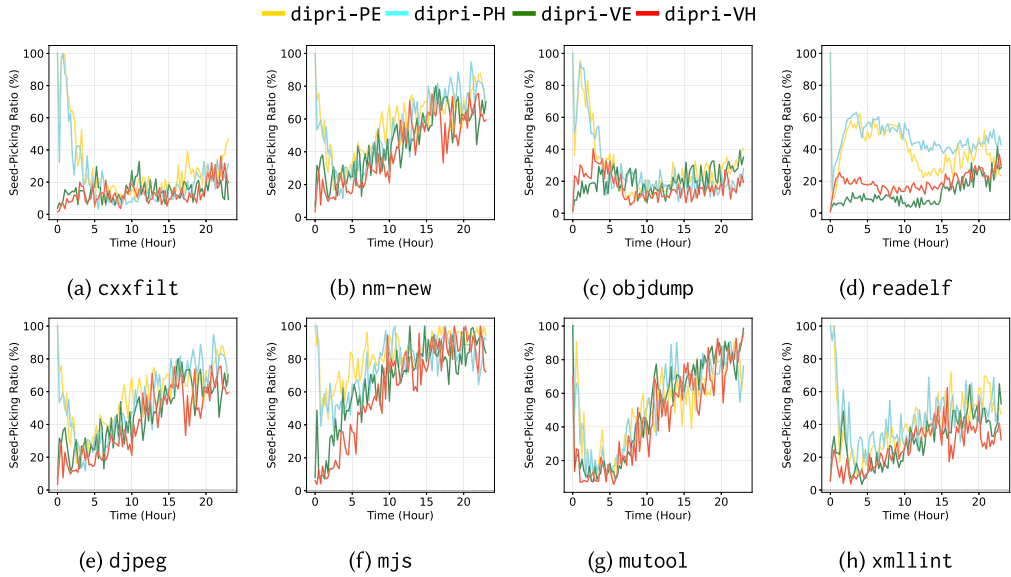


Fig. 8. The percentage of the seeds picked by dipri-PE, dipri-PH, dipri-VE, and dipri-VH during fuzzing.

5.1.4 SPRs of DiPRI under Different Configurations. The SPR refers to the ratio of the number of seeds being picked by DiPRI and the total number of the seeds within the seed queue at the time this round of prioritization is conducted. Suppose the i is the round of DiPRI prioritization and t_i is the timestamp at which the prioritization happens, n_i is the number of seeds picked after the round i prioritization finishes and before the round $i + 1$ starts, N_i is the total number of the seeds with the seed queue at round i , then the SPR at round i is calculated as $SPR(i, t_i) = n_i/N_i$. Figure 8

Table 4. Average SPRs (in Percentage) of dipri-PE, dipri-PH, dipri-VE, and dipri-VH

Target	dipri-PE	dipri-PH	dipri-VE	dipri-VH
cxxfilt	29.12	24.16	11.12	9.95
nm-new	67.99	62.27	36.76	33.21
objdump	31.91	31.10	17.40	20.33
readelf	39.84	47.12	8.84	17.73
djpeg	47.29	41.98	31.66	23.36
mjs	77.55	68.46	32.29	23.42
mutool	35.74	35.65	20.70	18.81
xmllint	39.75	44.19	23.61	19.21
Average	46.15	44.37	22.80	20.75

plots the average SPR of the four configurations under PERIODICAL and VANILLA modes, where the X axis represents the elapsed time and the Y axis represents the average SPR in percentage. We do not include the two configurations under ADAPTIVE mode in Figure 8, as the ADAPTIVE mode is designed to prioritize when all of the last prioritized seeds have been picked for input generation (see Section 3.4), so the SPRs of dipri-AE and dipri-AH are always 100%. As depicted in Figure 8, the tendencies of the configurations from the same prioritization mode are similar, whereas the tendencies of the configurations from different modes are different. Specifically, the curves of dipri-PE and dipri-PH usually first drastically drop from peaks in the first 5 hours and then slowly climb in the remaining time, whereas the curves of dipri-VE and dipri-VH present upward trends in general. The mode-specific differences in SPRs can be attributed to different designs of the two prioritization modes, where the VANILLA mode prioritizes whenever the seed queue has grown while the PERIODICAL prioritizes when the preset period δ (i.e., 1 minute in our experiments) is reached or all of the last prioritized seeds have been exhausted. When at the early stage of fuzzing (e.g., the first 5 hours), the capacity of the seed queue is typically small and thus can be easily exhausted by DiPRI under PERIODICAL mode within δ . However, as fuzzing proceeds and the seed queue enlarges, dipri-PE and dipri-PH are gradually unable to exhaust the seed queue and thereby turn to pick partial seeds. In addition to the tendencies, we extract the average SPRs of the four configurations to investigate the impact of the overall SPR, which are exhibited in Table 4. The exhibited average SPR values are calculated by first averaging the SPR of all rounds of DiPRI within a fuzz campaign and then averaging over 10 trials. Table 4 shows that (1) DiPRI tends to pick slightly fewer seeds with Hamming than with Euclidean, which establishes a 1.78% less of SPR under PERIODICAL and a 2.05% less of SPR under VANILLA on average, and (2) DiPRI picks more seeds under PERIODICAL than under VANILLA, which establishes 2.0 $\times$ and 2.1 $\times$ the SPR on Hamming and Euclidean. Recalling that DiPRI is designed to pick all queued seeds under ADAPTIVE and code coverage results described in Section 5.1.2, DiPRI is likely to get benefit via picking more queued seeds. These results imply that DiPRI may tend to rank the most powerful seeds at relatively backward positions rather than at the very forefront of the queue.

5.2 RQ2: Comparisons to Other Seed Prioritization Approaches

5.2.1 Prioritization Approaches. We take the two prioritization approaches provided by AFL++ as the baselines. By default, AFL++ adopts a weighted random prioritization based on an alias table [2]. When specified with -Z option in the command line, AFL++ turns to use the old order-of-queued approach previously adopted by AFL [1]. For clarity, we mark these two prioritization approaches as aflpp and aflpp-Z when describing the experimental results. As for DiPRI, we choose the dipri-VH and dipri-AH configurations, as (1) dipri-VH is the plainest configuration

Table 5. Average Branch Coverage (i.e., the Larger-Font Values on the Left of “/”) and the Coverage-Based Ranks (i.e., the Smaller-Font Values on the Right of “/”) That the Two Prioritization Approaches (i.e., afl++ and afl++-Z) and the Two DiPRI Configurations (i.e., dipri-AH and dipri-VH) Achieve on the Eight Targets from Real-World Projects (the Upper Part of the Table) and the Eight Targets from the FuzzBench Benchmark (the Lower Part of the Table) over 10 Trials of the 23-Hour Campaigns

Ranks on the Real-World Benchmark					
Target	afl	afl++	afl++-Z	dipri-AH	dipri-VH
cxxfilt	x	2780.9/2	2750.6/4	2782.6/1	2771.1/3
nm-new		2901.0/2	2871.6/3	2917.8/1	2836.6/4
objdump		5490.7/1	5385.1/2	5368.2/3	5273.4/4
readelf		11916.1/2	11749.1/4	11871.5/3	11971.6/1
djpeg		1989.8/2	1973.0/4	2061.7/1	1985.0/3
mjs		1548.6/3	1556.9/2	1601.9/1	1517.0/4
mutool		2800.0/1	2793.1/3	2784.1/4	2796.3/2
xmllint		5365.4/2	5263.2/3	6182.3/1	5203.8/4
Average Rank		1.9/1	3.1/3	1.9/1	3.1/3
Ranks on the FuzzBench Benchmark					
Target	afl	afl++	afl++-Z	dipri-AH	dipri-VH
bloaty_fuzz_target	6084.1/1	4884.1/4	5785.6/2	4945.5/3	4834.5/5
curl_curl_fuzzer_http	10336.5/4	10667.5/2	10754.9/1	10510.9/3	9813.8/5
freetype2_ftfuzzer	8088.2/4	9498.1/3	9646.5/2	10094.3/1	7732.2/5
jsoncpp_jsoncpp_fuzzer	515.9/5	520.0/1	519.9/2	519.7/3	519.6/4
lcms_cms_transform_fuzzer	1061.1/5	1881.1/4	2014.4/2	1927.4/3	2058.7/1
libpcap_fuzz_both	39.7/5	2925.5/4	3152.0/1	3003.1/3	3054.6/2
re2_fuzzer	2826.0/5	2876.6/1	2874.2/4	2875.2/3	2875.3/2
sqlite3_ossfuzz	18653.1/1	15235.6/3	18224.6/2	11925.9/5	14263.1/4
Average Rank	3.4/4	2.7/2	1.9/1	3.0/3	3.5/5

The “μRank” row displays the average values of the coverage-based rank on the eight targets, where the smaller-font values are the relative ranks. The results ranked first are colored in green, and the results ranked second are colored in yellow.

that can embody the basic characteristics of DiPRI, and (2) dipri-AH is the best configuration according to the results of the experiments on DiPRI configurations (see Section 5.1). We compare the aforementioned seed prioritization approaches on 24 fuzz targets—that is, 8 from real-world projects, 8 from FuzzBench, and 8 from Magma. In addition to these four approaches, we include AFL [1], which is marked as “afl” in figures and tables, on FuzzBench and Magma benchmarks.

5.2.2 Performance on Code Coverage. Figure 9 displays the tendencies of branch coverage that the studied seed prioritization approaches achieve across 10 of 23-hour campaigns, where Figure 9(a) through (h) are the fuzz targets from real-world projects, whereas Figure 9(i) through (p) are the fuzz targets from FuzzBench benchmark. Note that we use the prefix rather than the full name of FuzzBench targets to save space, and we will keep using prefixes to refer to these targets in the rest of this paragraph for conciseness. Table 5 exhibits the average final coverage and the coverage-based ranks for the studied seed prioritization approaches, where the upper half is about the real-world benchmark while the lower half is about the FuzzBench benchmark. With Figure 9 and Table 5, we find that dipri-AH is slightly inferior to afl++ and afl++-Z while dipri-AH is totally worse. The performances of dipri-AH and dipri-VH are better on the real-world benchmark than on FuzzBench benchmark. Specifically, dipri-AH ranks first on 5 out of 8 targets from the real-world projects, whereas afl++ ranks first on 2 out of 8 targets and ranks second on 5

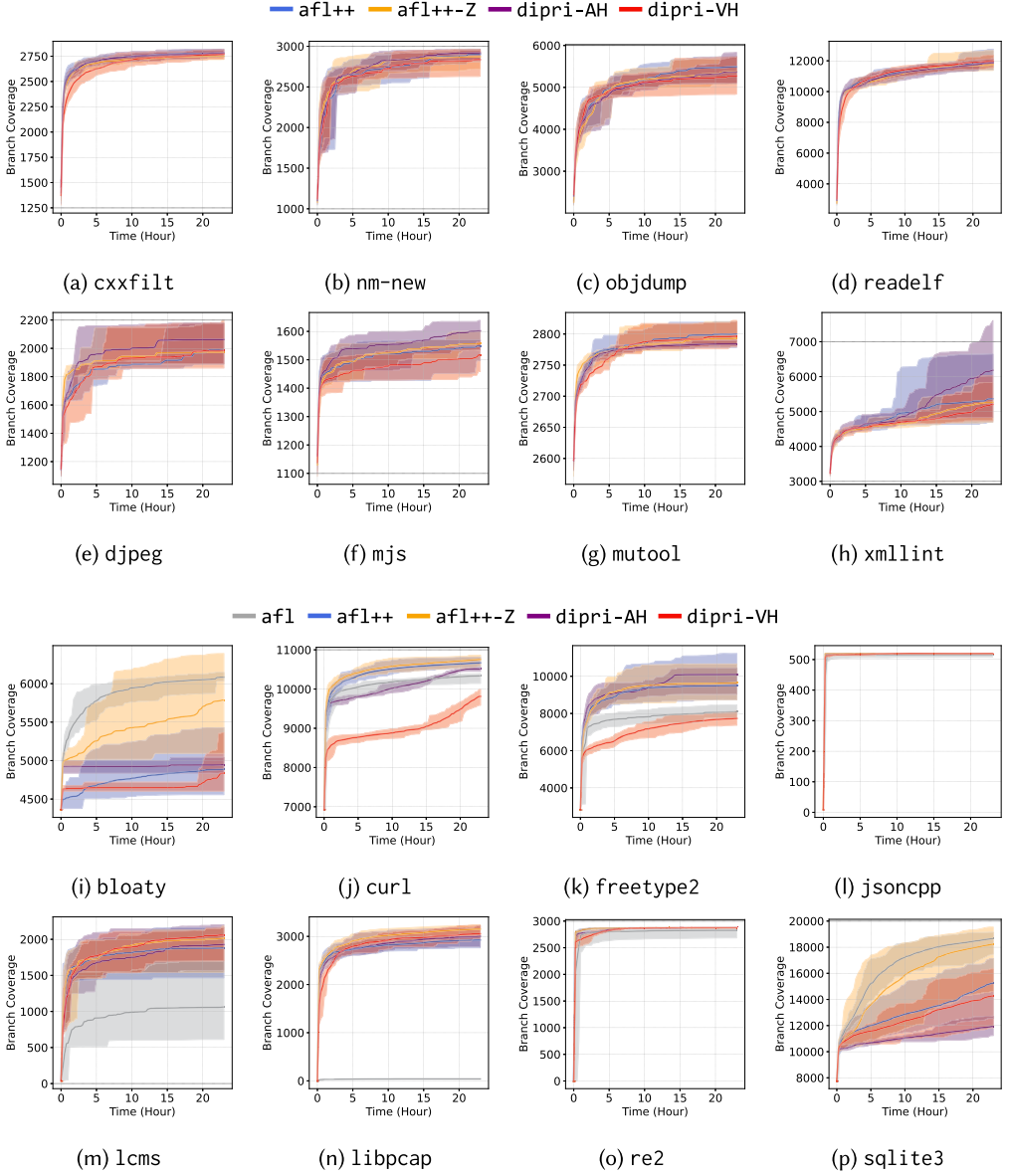


Fig. 9. The branch coverage that afl++, afl++-Z, dipri-VH, and dipri-AH achieve over 10 trials of the 23-hour campaign. The solid lines indicate the mean coverage, and the light-colored areas are ranges between the maximum and minimum coverage. The X axis is the elapsed time, whereas the Y axis is the branch coverage.

out of the 6 remaining targets. Both dipri-AH and afl++ get a relative rank of first and both dipri-VH and afl++-Z get a relative rank of third on the real-world benchmark. The results appearing on the FuzzBench benchmark are quite different from the ones on the real-world benchmark. The dipri-AH configuration is slightly worse than afl++ and gets a middle relative rank of third. The dipri-VH configuration is significantly inferior to dipri-AH, afl++, and afl++-Z on

targets like bloaty, curl, and freetype2, and ranks last among the five studied seed prioritization approaches. However, although performing the worst concerning the relative coverage-based rank, dipri-VH still achieves a rank of first on lcms and a rank of second on libpcap and re2, indicating its commendable capability of covering code on some targets. One possible interpretation for the degraded performance of DiPRI on FuzzBench targets is the usage of the initial seeds, which is critical to the performance of mutation-based fuzzers [30, 62]. As in the setups of fuzz targets described in Section 4.2, we prepare exactly one initial seed for each of the targets from the real-world benchmark while using standard initial seed corpora for targets from FuzzBench, most of which contain multiple and more complex seed files. Given a larger initial seed queue and more complex seed files, DiPRI naturally has to spend more time evaluating seed distances at an early stage of fuzzing, which may reduce test throughput and impair DiPRI exploring new code regions. The most surprising results are that afl++-Z performs the best on FuzzBench targets, and it ranks first or second on 7 out of 8 targets and gets a relative rank of first. In fact, not to mention afl++-Z, even the trivial afl, which is generally considered outdated and is equipped with the same order-of-queued seed prioritization approach as afl++-Z, can beat the other three seed prioritization approaches hosting on AFL++ on targets like bloaty and sqlite3. The results suggest that a complex prioritization approach may instead degrade the host fuzzers under certain scenarios. Figure 10 visualizes the distributions of the final branch coverage. Although being slightly worse than afl++ in terms of average branch coverage on FuzzBench targets, dipri-AH is better than afl++ in terms of median branch coverage on 11 out of 16 targets, indicating that the final branch coverage of dipri-AH is skewed to be higher.

5.2.3 Performance on Finding Bugs. Table 6 shows the summary of the experimental results on the Magma benchmark, where the leftmost five columns are results of the five studied seed prioritization approaches, namely afl, afl++, afl++-Z, dipri-AH, and dipri-VH. Due to space limitations, we leverage T17 through T24, the TIDs displayed in Table 1, to distinctly mark the targets from the Magma benchmark. Table 6 contains the statistics for reached and triggered bugs, which respectively are the total number of the occurrences of bugs across all 10 trials (“Tot.”), the number of unique bugs (“Uni.”), the number of missed bugs (“Missed”), the number of bugs that an approach reaches/triggers with the least time (“Fastest”), the number of bugs that an approach reaches/triggers in most trials (“Most Steady”), and the consistency of each approach (“Consistency”). Except for “Most Steady,” we conduct other statistic items following Luo et al. [42], where the “Consistency” is calculated as “Sum of Tot./Sum of Uni./# of Trials.” Note that instead of directly using the bug IDs provided by Magma [32], we concatenate Magma IDs and target names to uniquely identify bugs, as a fuzzer may reach/trigger a bug with the same Magma ID through different fuzz targets from the same project (e.g., the xmllint and xml_read_memory_fuzzer targets from the libxml2 project). The upper half of Table 6 displays the statistics of reached bugs and the lower half displays the statistics of triggered bugs, where the best results are highlighted in green. According to the results presented in Table 6, dipri-VH reaches the most unique bugs on six out of eight targets, whereas afl++-Z achieves the largest “Tot.” values on six out of the eight targets; these two approaches also perform the best on “Sum” values of “Uni.” and “Tot.,” respectively. Notably, although dipri-AH and dipri-VH are worse than afl++ and afl++-Z on some of the targets in terms of “Tot.,” they can be equal to or even better than afl++ and afl++-Z on these targets in terms of “Uni.,” which implies DiPRI’s capability of exploring remote code regions. When it comes to the triggered bugs, dipri-AH comes into first place on all eight targets, whereas afl++-Z achieves the largest “Tot.” values. Moreover, afl++ triggers the most bugs fastest while being a bit unstable compared to afl++ and afl++-Z in terms of “Most Steady” and “Consistency.” However, as another configuration of DiPRI, dipri-VH does not perform as well as dipri-VH on

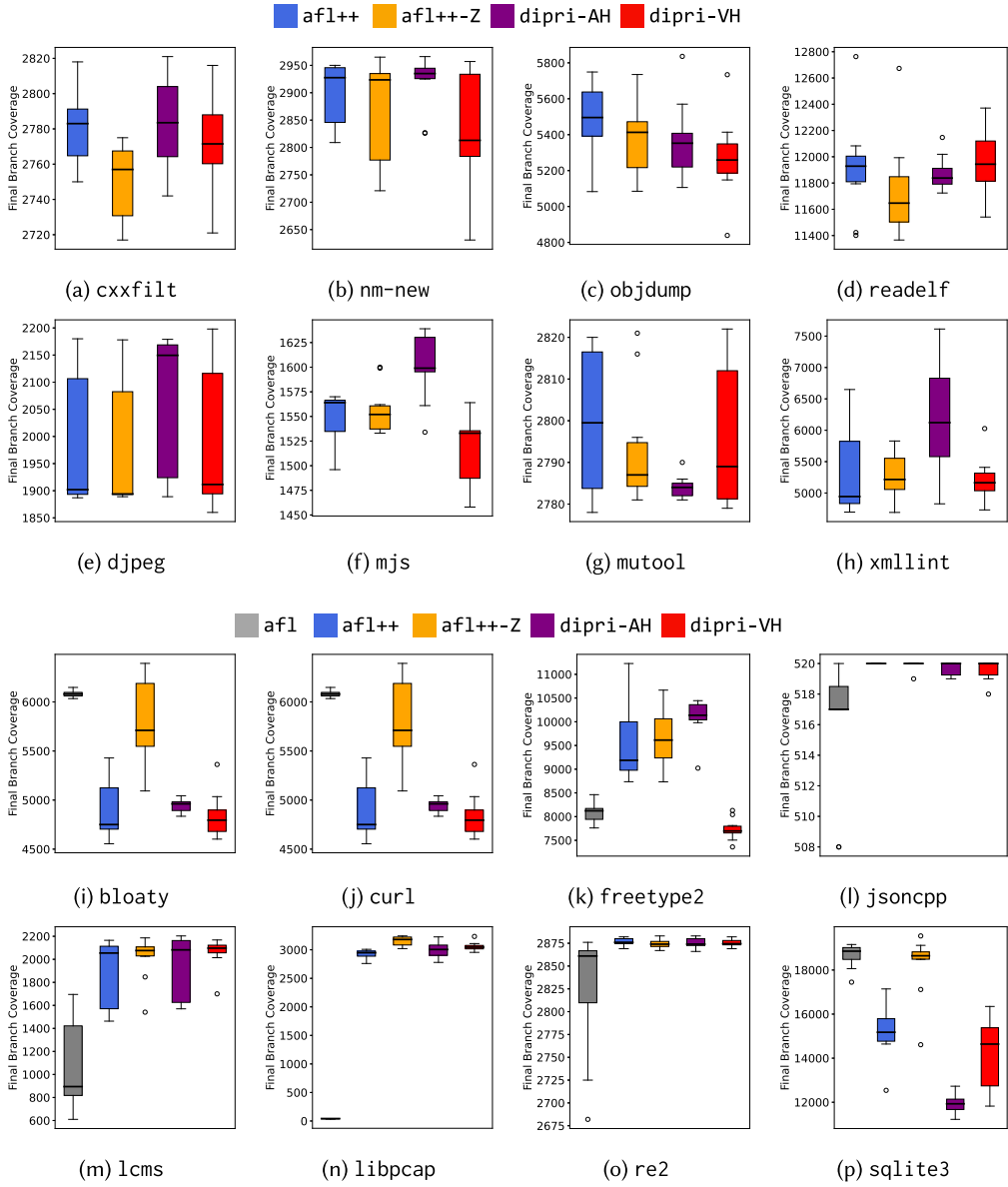


Fig. 10. The distributions of the final branch coverage that afl++, afl++-Z, dipri-VH, and dipri-AH achieve at the end of each trial of the 23-hour campaign. The Y axis is the final branch coverage, and each box visualizes the results of an approach. The short bold lines within the boxes denote median final branch coverage.

triggering bugs, which not only misses seven unique bugs but also fails in terms of “Fastest” and “Most Steady.” Recalling the results that dipri-VH performs the best on reaching bugs and the technical differences between VANILLA and ADAPTIVE modes, the inferiority of dipri-VH on triggering bugs indicates that the frequent executions of prioritization may impair DIPri’s effectiveness. To further analyze the overlaps of Magma bugs reached/triggered by the five studied approaches, we

Table 6. Summary of the Bug-Finding Results on the Magma Benchmarks across 10 Trials of the 23-hour Campaigns

Statistics of Reached Bugs												
Appr.	afl		afl++		afl++-Z		dipri-AH		dipri-VH		k-sched.	
	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.
T17	60	6	60	6	60	6	60	6	60	6	60	6
T18	80	8	80	8	80	8	80	8	80	8	80	8
T19	71	8	66	9	61	7	63	7	56	8	52	8
T20	10	1	33	5	37	5	32	5	25	6	3	2
T21	70	7	70	7	71	8	72	8	74	9	70	7
T22	10	1	24	4	26	4	24	5	17	2	10	1
T23	10	1	19	2	19	2	17	2	14	2	11	2
T24	69	10	132	15	133	16	126	15	119	16	49	9
Sum	380	42	484	56	487	56	474	56	445	57	335	43
Missed	42		28		28		28		27		41	
Fastest	17		19		30		18		16		1	
Most Steady	32		47		48		44		40		29	
Consistency	0.90		0.86		0.87		0.85		0.78		0.78	
Statistics of Triggered Bugs												
Appr.	afl		afl++		afl++-Z		dipri-AH		dipri-VH		k-sched.	
	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.	Tot.	Uni.
T17	19	2	27	3	28	3	30	3	30	3	12	2
T18	54	6	70	7	70	7	70	7	70	7	39	6
T19	31	4	41	5	36	5	40	5	33	4	22	4
T20	10	1	29	3	33	4	30	4	20	4	1	1
T21	11	2	27	3	29	4	32	5	31	4	10	1
T22	0	0	14	3	16	3	13	4	10	1	0	0
T23	10	1	10	1	10	1	10	1	7	1	10	1
T24	7	3	42	7	46	6	42	8	36	6	3	2
Sum	142	19	260	32	268	33	267	37	237	30	97	17
Missed	65		52		51		47		54		67	
Fastest	22		27		27		32		24		22	
Most Steady	10		26		28		27		20		7	
Consistency	0.75		0.81		0.81		0.72		0.79		0.57	

The upper part of the table exhibits the statistics of reached bugs, and the lower part exhibits the statistics of triggered bugs. The “Tot.” columns display the total number of times that bugs are reached/triggered by different approaches, whereas the “Uni.” columns display the number of unique bugs reached/triggered. The first through eighth rows show the results on different fuzz targets, where T17 through T24 are target IDs consistent with Table 1. The “Sum” rows present the sum number of bugs on all eight targets. The “Missed” rows present the number of missed bugs. The “Fastest” rows present the number of bugs that the studied approaches reach/trigger with the least time, whereas the “Most Steady” rows present the number of bugs that the studied approaches reach/trigger in most trials. The “Consistency” rows show the consistency of the studied approaches, which is calculated as “Sum of Tot./Sum of Uni./# of Trials.” The best results are highlighted in green.

visualize the results of unique bugs’ Venn diagrams, which are illustrated in Figure 11. The diagram of reached bugs is on the left (see Figure 11(a)), whereas the diagram of triggered is on the right (see Figure 11(b)). The colors indicating the five studied approaches are tagged at the top of the figure. According to Figure 11(a), the bugs reached by the five approaches are overlapped in the most part while complementary in small parts, of which three (i.e., two by dipri-VH and one by dipri-AH) can only be reached by DiPRI. As for triggered bugs, the overlaps presented in Figure 11(b)

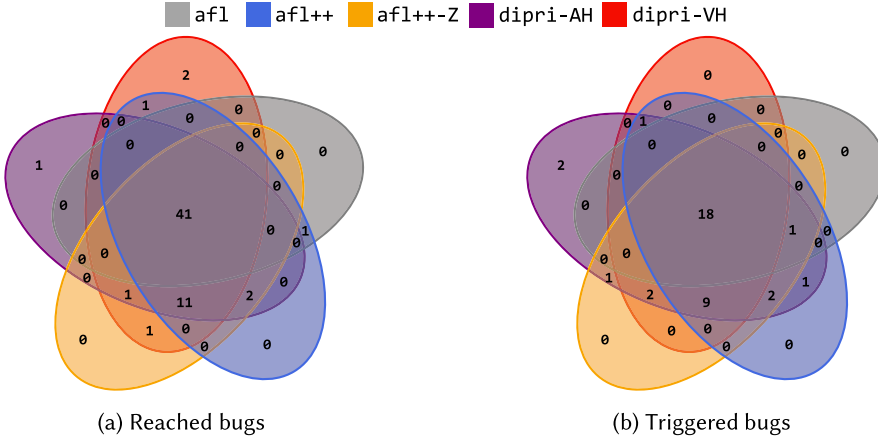


Fig. 11. Venn diagrams depicting the overlaps of the Magma bugs reached/triggered by afl, afl++, afl++-Z, dipri-AH, and dipri-VH. The colors corresponding to each of the approaches are tagged on top of the diagrams.

are similar to those presented in Figure 11(a), which also includes a major overlap achieved together by the five approaches and several minor overlaps. The most important difference between Figure 11(a) and (b) is that dipri-AH basically covers all of the Magma bugs that are triggered by the other four approaches. Furthermore, dipri-AH additionally triggers two unique bugs that the other four approaches fail to trigger. The results demonstrate that DiPRI can facilitate the host fuzzers with the capability of finding bugs.

5.3 RQ3: Comparisons to SOTA Seed Scheduling Techniques

5.3.1 SOTA Seed Scheduling Techniques. Seed scheduling is another field highly relevant to seed prioritization, which typically involves the selection of seed inputs from the seed pool and the adjustment of seed energy. In this section, we compare DiPRI to two SOTA seed scheduling techniques, namely K-Scheduler [66] and Alphuzz [80], under dipri-AH and dipri-VH configurations to explore whether DiPRI is competitive. K-Scheduler is a data-driven technique that prioritizes the seeds close to the boundary of visited and unvisited code regions with the help of centrality analysis. Alphuzz is a novel technique that leverages the mutation relationships among seeds as the basis of seed scheduling. We compare DiPRI with the baseline seed scheduling techniques on both coverage-based and bug-based benchmarks. For coverage-based benchmarks, we build the eight targets from real-world projects with the official guidelines provided by the authors of K-Scheduler and Alphuzz. We successfully build four targets (i.e., cxxfilt, nm-new, readelf, and mjs) for K-Scheduler and all eight targets for Alphuzz. For bug-based benchmarks, we merely compare DiPRI to K-Scheduler on the Magma benchmark because (1) Magma provides off-the-shelf integration of K-Scheduler, and (2) we fail to integrate Alphuzz into Magma with the code and guidelines provided by the authors of Alphuzz [5].

5.3.2 Performance on Code Coverage. Figure 12 display the tendencies of branch coverage achieved by K-Scheduler, Alphuzz, and the two DiPRI configurations over 10 trials of 23-hour campaigns, whereas Figure 13 display the distributions of branch coverage. Different from other figures reporting coverage results, we adjust the display sequences of targets in Figure 12 to gather the four targets successfully built with K-Scheduler in the first four subfigures (see Figure 12(a)–(d)) and put the four targets without K-Scheduler in the last four (see Figure 12(e) through (h)). According to

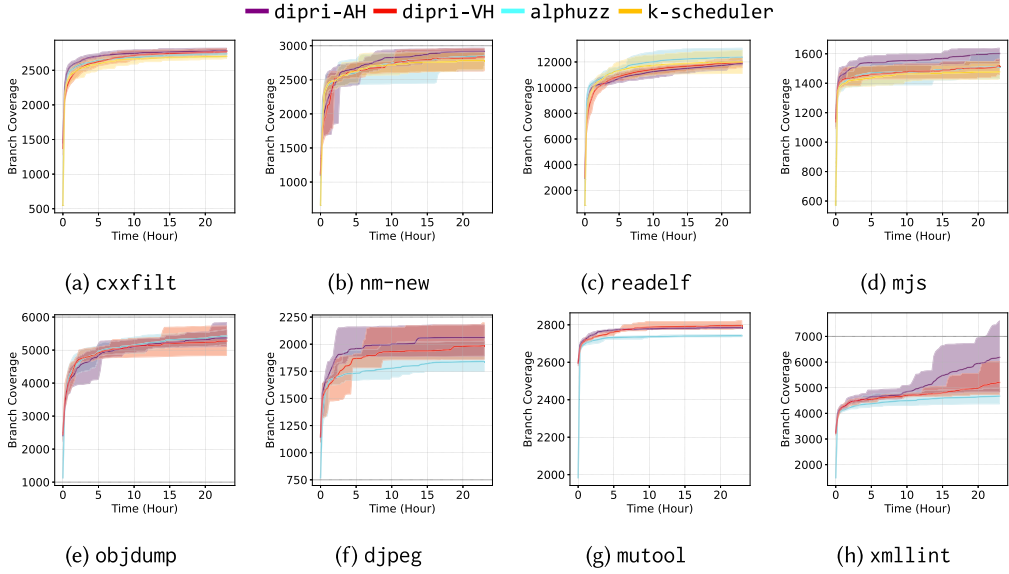


Fig. 12. The branch coverage that dipri-VH, dipri-AH, alphuzz, and k-scheduler achieve over 10 trials of the 23-hour campaign. The solid lines indicate the mean coverage, and the light-colored areas are ranges of coverage. The X axis is the elapsed time, whereas the Y axis is the branch coverage.

Figure 12, DiPRI outperforms Alphuzz on six out of eight fuzz targets under dipri-AH, while performing closely to Alphuzz by beating and failing to Alphuzz on three and two fuzz targets under dipri-VH. The results demonstrate that, in terms of average branch coverage, DiPRI is better than Alphuzz under dipri-AH and comparable to Alphuzz under dipri-VH, which further confirms the superiority of ADAPTIVE. Compared to K-Scheduler, DiPRI is better on three of the four targets except for readelf, where the two DiPRI configurations and K-Scheduler achieve close final coverage and K-Scheduler is slightly better. The results show that DiPRI is superior to K-Scheduler in terms of average branch coverage. Considering Figure 13, dipri-AH outperforms Alphuzz and K-Scheduler on six out of eight targets in terms of median coverage except for readelf and objdump, whereas dipri-AH is additionally inferior to Alphuzz in terms of median coverage on nm-new. The results unveil that DiPRI is competitive with SOTA seed scheduling techniques in terms of code coverage.

5.3.3 Performance on Finding Bugs. The three rightmost columns of Table 6 exhibit the statistics of the bugs that the two DiPRI configurations and K-Scheduler find on the Magma benchmark. The statistics show that the two DiPRI configurations present overall superiority over K-Scheduler on all eight targets in terms of both the number of reached and triggered bugs. Most strikingly, DiPRI reaches 33% more unique bugs than K-Scheduler under dipri-VH configuration and triggers 118% more unique bugs than K-Scheduler under dipri-AH configuration. Besides the number of bugs, DiPRI outperforms K-Scheduler with respect to bug-finding speed and stability. On the one hand, in terms of bug-finding speed, DiPRI fastest reaches $18.0\times$ and $16.0\times$ bugs and fastest triggers $1.5\times$ and $1.1\times$ bugs compared to K-Scheduler under dipri-AH and dipri-VH configurations. On the other hand, in terms of bug-finding stability, DiPRI reaches $1.5\times$ and $1.4\times$ bugs and triggers $3.9\times$ and $2.9\times$ bugs in most trials compared to K-Scheduler under dipri-AH and dipri-VH configurations; moreover, it achieves $1.3\times$ and $1.4\times$ consistency compared to K-Scheduler on triggering bugs under the two studied configurations. Figure 14 further visualizes the overlaps of Magma bugs

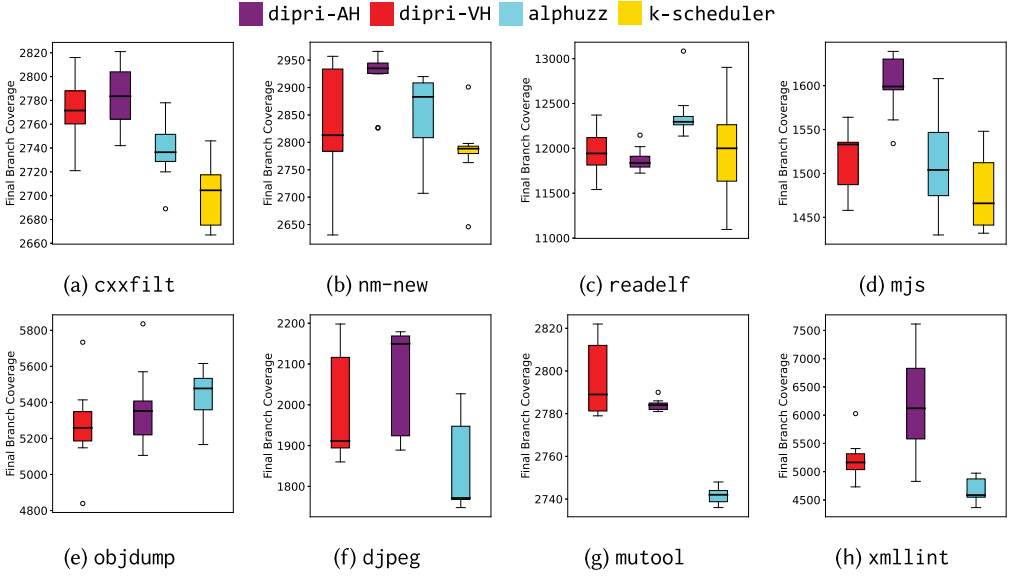


Fig. 13. The distributions of the final branch coverage that dipri-VH, dipri-AH, alphuzz, and k-scheduler achieve at the end of each trial of the 23-hour campaign. The Y axis is the final branch coverage, and each box shows the results of an approach. The short bold lines within the boxes denote median final branch coverage.

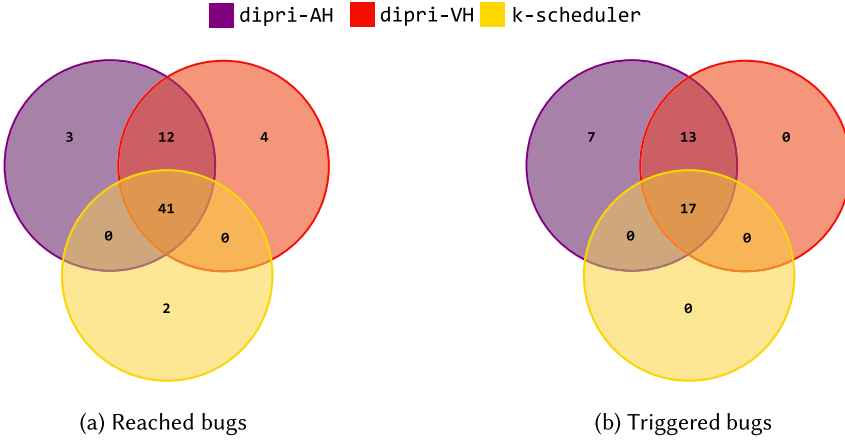


Fig. 14. Venn diagrams depicting the overlaps of the Magma bugs reached/triggered by dipri-AH, dipri-VH, and k-scheduler. The colors corresponding to each of the approaches are tagged atop the diagrams.

reached (see Figure 14(a)) and triggered (see Figure 14(b)) by dipri-AH, dipri-VH, and K-Scheduler across 10 trials of 23-hour campaigns. As presented in Figure 14, DIPRI can reach almost all of the bugs that K-Scheduler reaches in 10 trials except for 2 bugs; furthermore, it not only triggers all of the 17 bugs triggered by K-Scheduler but additionally triggers 20 bugs, where 7 bugs are merely triggered by dipri-AH. The results demonstrate that DIPRI is superior to K-Scheduler in terms of both bug counts and bug types.

5.4 RQ4: Effectiveness of Distance-Based Seed Evaluation

5.4.1 Metrics for Seed Evaluation. DiPRI relies on two components to conduct seed prioritization, namely distance-based seed evaluation and seed ordering. With the two-component design, a natural question is raised: what if the evaluation metric is switched from the distance-based one to a metric of another type, can the host fuzzer still get benefits from seed reordering? To independently evaluate the contributions of the proposed distance-based metric and seed reordering, we run DiPRI with other evaluation metrics and compare the results produced by DiPRI under the `dipri-VH` and `dipri-AH` configurations. The compared metrics are chosen from the ones that the host fuzzer (i.e., AFL++ in our evaluation) employs for power scheduling or corpus minimization [3, 30]. For clarity, we refer to such metrics as *intrinsic metrics*. The details of the intrinsic metrics are as follows:

- *Coverage*: AFL++ records the coverage attained by each (seed) test input with a field named `bitmap_size`. A seed with bigger `bitmap_size` is favored and tends to be assigned more energy at power scheduling. Following the design of AFL++, we leverage `bitmap_size` as priority scores to rank the seeds that achieve higher coverage through seed reordering.
- *Path depth*: AFL++ holds the assumption that the (seed) test inputs with deeper paths are more likely to reveal rare program behaviors. As a result, it assigns seeds with larger depth values with more energy to generate more offspring inputs from them. In alignment with AFL++'s assumption, we attach seeds with larger depth values with higher-priority scores.
- *Handicap*: According to the document of AFL++, the handicap profiles the difficulty in learning the path executed by a seed, where the latecomer seeds are attached with higher handicap values and assigned more energy. Therefore, similar to coverage, we view handicap values as priority scores and rank seeds with larger handicap values higher when performing seed reordering.
- *Input size*: AFL++ prefers (seed) test inputs smaller in size, as they can be fuzzed more efficiently. When scanning the given initial seeds, AFL++ tends to recommend the testers provide smaller files under 1 KB [2]. Following the preference of AFL++, we leverage the reciprocals of the input sizes as priority scores when performing seed reordering to rank smaller seeds higher.
- *Execution time*: AFL++ records the time consumed by each (seed) test input in microseconds with a field named `exec_us`. When conducting power scheduling, AFL++ prefers faster seeds and tends to assign them more energy. Following the design of AFL++, we leverage the reciprocals of `exec_us` as priority scores to rank faster seeds higher through seed reordering.

5.4.2 Effectiveness of Different Seed Evaluation Metrics. We compare the effectiveness of different seed evaluation metrics with branch coverage on the eight targets from real-world projects. We also include AFL++, which adopts the default random-based seed prioritization approach, in this comparison to explore whether the intrinsic-based seed evaluation can facilitate plain AFL++. Table 7 shows the ranks of seed evaluation metrics in terms of average final branch coverage and Figure 15 shows the tendencies of branch coverage over 15 trials of 23-hour campaigns. According to Table 7 and Figure 15, `dipri-AH` still comes into the first place on the eight targets from real-world projects, whereas the plain AFL++ is slightly behind. The most noticeable result is that `dipri-AH` presents far better than `cov`, which indicates that simply prioritizing with a value of coverage is ineffective, protruding the necessity of calculating seed distances with coverage vectors. Although inferior to `dipri-AH` and AFL++ in terms of average branch coverage, the five intrinsic metrics can perform well on some of the targets. Specifically, `depth` performs the best on `cxxfilt` and `djpeg`, and `handi` gets the first and second place on `readelf` and `xmllint`,

Table 7. Average Branch Coverage (i.e., the Larger-Font Values on the Left of “/”) and the Coverage-Based Ranks (i.e., the Smaller-Font Values on the Right of “/”) That afl++, dipri-VH, dipri-AH, and the Five Intrinsic-Metric-Powered Prioritization Approaches Achieve on the Eight Targets from Real-World Projects over 10 Trials of the 23-Hour Campaigns

Target	afl++	AH	VH	cov	depth	handi	size	time
cxxfilt	2780.9 _{/3}	2782.6 _{/2}	2771.1 _{/5}	2756.1 _{/7}	2794.6 _{/1}	2780.5 _{/4}	2752.7 _{/8}	2768.2 _{/6}
nm-new	2901.0 _{/2}	2917.8 _{/1}	2836.6 _{/8}	2845.3 _{/7}	2856.1 _{/6}	2886.6 _{/3}	2884.1 _{/4}	2879.3 _{/5}
objdump	5490.7 _{/2}	5368.2 _{/3}	5273.4 _{/4}	4994.7 _{/8}	5264.4 _{/5}	5236.6 _{/6}	5152.3 _{/7}	5670.1 _{/1}
readelf	11916.1 _{/3}	11871.5 _{/4}	11971.6 _{/2}	11532.4 _{/5}	11473.0 _{/6}	11979.2 _{/1}	10953.1 _{/8}	11335.7 _{/7}
djpeg	1989.8 _{/6}	2061.7 _{/3}	1985.0 _{/7}	2073.4 _{/2}	2083.5 _{/1}	2003.2 _{/5}	2025.0 _{/4}	1956.8 _{/8}
mjs	1548.6 _{/5}	1601.9 _{/2}	1517.0 _{/8}	1564.2 _{/3}	1530.8 _{/7}	1549.8 _{/4}	1546.3 _{/6}	1607.9 _{/1}
mutool	2800.0 _{/1}	2784.1 _{/8}	2796.3 _{/2}	2794.0 _{/3}	2789.2 _{/6}	2789.3 _{/5}	2791.7 _{/4}	2786.5 _{/7}
xmllint	5365.4 _{/5}	6182.3 _{/1}	5203.8 _{/6}	5029.7 _{/7}	4890.9 _{/8}	5577.6 _{/2}	5521.6 _{/3}	5500.4 _{/4}
Average Rank	3.4 _{/2}	3.0 _{/1}	5.2 _{/6}	5.2 _{/6}	5.0 _{/5}	3.8 _{/3}	5.5 _{/8}	4.9 _{/4}

The “ μ Rank” row displays the average values of the coverage-based rank on the eight targets, where the smaller-font values are the relative ranks. The results ranked first are colored in green, and the results ranked second are colored in yellow.

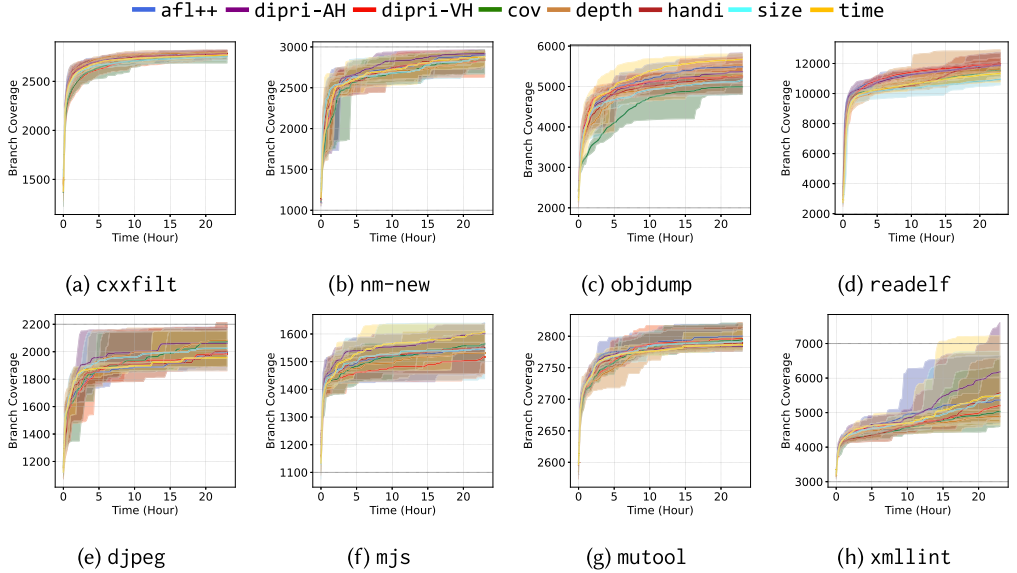


Fig. 15. The branch coverage achieved by eight prioritization approaches (i.e., afl++, dipri-AH, dipri-VH and DIPRI with the five intrinsic evaluation metrics) over 10 trials of the 23-hour campaign. The solid lines indicate the mean coverage, and the light-colored areas visualize the ranges between the maximum and minimum coverage. The X axis is the elapsed time, whereas the Y axis is the branch coverage.

respectively, whereas time comes in first on objdump and mjs. These target-specific best results imply that depth, handi, and time have the potential to boost greybox fuzzing by acting as seed evaluation metrics. The dipri-VH ranked sixth among the eight considered seed evaluations, which fights a draw with cov and loses to depth, handi, and time. Among the five studied intrinsic metrics, handi comes in the best with an absolute rank of 3.8, whereas size comes in the worst with an absolute rank of 5.5. These results indicate that the latecomer seeds are promising for new coverage, which conforms to the document of AFL++, whereas the small-sized seeds are not prone

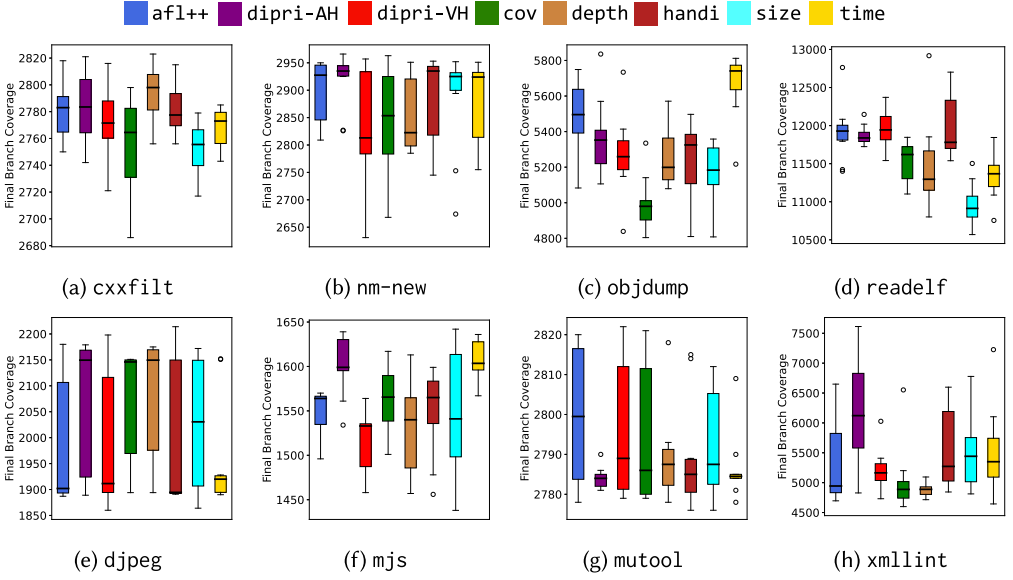


Fig. 16. The distributions of the final branch coverage that eight prioritization approaches achieve at the end of each trial of the 23-hour campaign. The eight approaches include the default random-based approach adopted by AFL++ (i.e., afl++), two DiPRI configurations (i.e., dipri-VH and dipri-AH), and five prioritization approaches powered by intrinsic seed evaluation metrics (i.e., cov, depth, handi, size, and time). The Y axis is the final branch coverage, and each box visualizes the results of an approach. The short bold lines within the boxes denote median final branch coverage.

to giving birth to interesting offspring inputs. However, in contrast to size, the results of time are much better with an absolute rank of 4.9, which implies that (1) the small-sized seeds may not necessarily be executed fast, and (2) the fast seeds are more valuable than small-sized seeds for input generation. Figure 16 visualizes the distributions of the finally reached branch coverage of the eight studied seed evaluation approaches. According to Figure 16, the boxes of time tend to be more focused than over four intrinsic metrics on almost all eight targets except for nm-new, which implies that time may be the most steady intrinsic metric.

6 DISCUSSION

In this section, we further discuss the potential and limitations of DiPRI. We start by presenting the scalability of DiPRI with empirical data on the Java benchmark (Section 6.1) and then describe possible future extensions of DiPRI (Section 6.2). We end up discussing the threats to validity (Section 6.3).

6.1 Scalability of DiPRI

6.1.1 Scaling DiPRI to Java. We implement DiPRI on top of Zest [54], a semantic greybox fuzzer resided in the Java fuzzing platform JQF [53], and conduct a contrast experiment with Zest and six DiPRI configuration on the five programs provided in JQF to investigate how DiPRI scales to the fuzz targets implemented in another language. The information of the fuzzed Java programs is shown at Table 8. Note that we do not prepare seeds for the Java benchmark, as Zest is a generator-based fuzzer that can synthesize test input from scratch [51, 54]. The same as in Section 4.2.5, we fuzz each program for 23 hours and repeat each campaign for 10 trials. The experiment on the Java

Table 8. Information of the Java Benchmark

TID	Target	Artifact ID	Version	Input Type
T25	ant	org.apache.ant.ant	1.10.2	XML
T26	bcel	org.apache.bcel.bcel	6.2	Java Bytecode
T27	chess	org.lichess.scalachess_2.12	8.6.8	String
T28	closure	com.google.javascript.closure-compiler	v20180204	JavaScript
T29	rhino	org.mozilla.rhino	1.7.8	JavaScript

Table 9. Coverage-Based Ranks on the Java Benchmark

Target	zest	AE	AH	PE	PH	VE	VH
ant	1853.8 _{/1}	1837.9 _{/3}	1804.5 _{/5}	1841.0 _{/2}	1832.4 _{/4}	146.2 _{/7}	1585.7 _{/6}
bcel	2668.2 _{/4}	2668.4 _{/3}	2661.8 _{/5}	2698.4 _{/1}	2677.1 _{/2}	2421.9 _{/7}	2584.2 _{/6}
chess	3068.9 _{/2}	3068.2 _{/3}	3067.5 _{/4}	3069.1 _{/1}	3062.7 _{/5}	2870.8 _{/7}	3037.2 _{/6}
closure	15564.1 _{/3}	15721.9 _{/1}	15657.2 _{/2}	11520.0 _{/6}	14949.9 _{/5}	9758.6 _{/7}	14984.0 _{/4}
rhino	5781.5 _{/3}	5825.5 _{/2}	5848.8 _{/1}	5438.6 _{/6}	5684.2 _{/4}	4976.0 _{/7}	5629.0 _{/5}
Average Rank	2.6 _{/2}	2.4 _{/1}	3.4 _{/4}	3.2 _{/3}	4.0 _{/5}	7.0 _{/7}	5.4 _{/6}

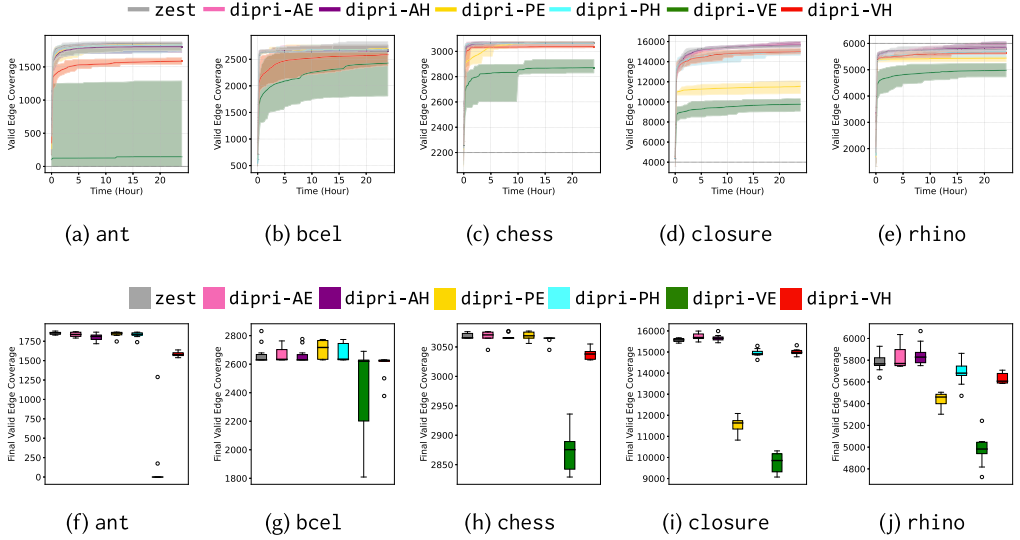


Fig. 17. The valid eight coverages that Zest and the six DIPRI configurations achieve across 10 trials of the 23-hour campaign. Parts (a) through (e) display the tendencies of valid edge coverage during fuzz campaigns, whereas parts (f) through (j) display the distributions of final valid edge coverage.

benchmark is conducted on a machine with 16 cores and 32 GB of memory running the Ubuntu 22.04 operating system.

6.1.2 Code Coverage Results on the Java Benchmark. Similar to Section 5, we also visualize the tendencies and distributions of code coverage, and rank Zest and the six DIPRI configurations with respect to average final code coverage. Note that for the Java benchmark, we leverage valid edge coverage as the code coverage indicator following the setup of Padhye et al. [54]. Figure 17 shows the coverage results of the studied approaches, where Figure 17(a) through (e) are the tendencies of valid edge coverage and Figure 17(f) through (j) are the distributions of final valid edge coverage, and Table 9 exhibits the coverage-based ranks. With Figure 17 and Table 9, we observe that

(1) `dipri-AE` and `zest` are the top two approaches in terms of code coverage, where `dipri-AE` is better with an insignificant improvement; (2) `dipri-AH`, `dipri-PE`, and `dipri-PH` perform slightly worse `Zest`; (3) except for `dipri-VH` and `dipri-VE`, the Euclidean distance measure performs better than the Hamming distance measure for other configurations under the same prioritization modes, which is exactly opposite to results that appeared on the C/C++ benchmark (Section 5.1.2); and (4) the `VANILLA` mode also performs the worst among the three proposed prioritization modes, where `dipri-VE` performs significantly worse than the other five configurations. Since `Zest` adopts an order-of-queued seed prioritization approach, the first and second findings imply that an over-complex seed prioritization and a partial seed picking may impair the performance of greybox fuzzing, which is similar to the results that appeared on the eight targets from `FuzzBench` (Section 5.2.2). The third finding implies that the Euclidean distance is more suitable than Hamming distance for Java targets, which may be attributed to the different natures of targets and host fuzzers. The last finding implies that a frequent execution of prioritization and a very small SPR can degrade the performance of `DiPRI`, which is consistent with the results we find in Section 5.1.4. The bad performance of `dipri-VH` on `ant` is a concrete example of the last finding. In particular, we parse the running log of `dipri-VH` and find that it always ranks a semantic-invalid seed first and repeatedly picks it for input generation, making itself trapped at limited code regions. Such a result also enlightens us to pay attention to the directions of (coverage-based) distances, as an outlier seed may not necessarily be good if it only covers a few program elements (e.g., branches and edges) and achieves a sparse coverage vector. We discuss this point further in Section 6.2.

6.1.3 Bug-Finding Results on the Java Benchmark. We parse the `fuzz.log` files recorded by `Zest` to extract the bugs triggered during fuzzing. Table 10 summarizes the bug-finding results on the Java benchmark, where the upper half of the table displays the statistics of bug counts and the lower half of the table displays the statistics of bug consistencies. The best results are highlighted in green. The calculation of bug consistency is analogous to the one we adopt for `Magma` bugs and is also described in the table caption. The “Bug” column displays the shorthand mnemonics of the bugs, which are essentially Java exceptions following the setup of Padhye et al. [54]. The details of these Java exceptions are described in the note to the table. With Table 10, we get three findings. First, `DiPRI` improves the total counts of bugs, such as the `NPE` from `closure` and the `ISE`, `NPE`, and `CCE` from `rhino`, indicating that `DiPRI` can drive the host fuzzer to exercise crashing code regions more rather than staying around at benign regions. Second, `DiPRI` can help `Zest` find bugs more consistently, such as the `CCE` and `AIOOB` from `rhino`. In particular, we notice that `dipri-PH` can consistently find the `AIOOB` from `rhino` with a consistency of 0.9, but can find the `CCE` from `rhino` with a consistency of only 0.3. This result implies that `Zest` may explore very different code regions under its original configuration and the `dipri-PH` configuration. Third, although `DiPRI` facilitates `Zest` in terms of the bug counts and bug consistencies under configurations like `dipri-AE`, `dipri-AH`, and `dipri-PH`, it can also refrain `Zest` from discovering bugs like the `ISE` from `closure`. Meanwhile, considering that `Zest` only finds the `ISE` from `closure` once across thousands of hundreds of executions, the bad impacts that `DiPRI` can bring to `Zest` are pretty accidental and insignificant.

6.1.4 Time Overhead on the Java Benchmark. Figure 18 shows the time overhead of `DiPRI` on Java benchmarks, where Figure 18(a) through (e) are the tendencies of time occupancy during 23-hour fuzzing and Figure 18(f) through (j) are the distributions of time occupancy at the end of campaigns. In contrast to the close time overheads appearing on the C/C++ benchmark (Section 5.1.3), the time occupancy of `DiPRI` appears to be inconsistent on the Java benchmark. First, the time overheads of `dipri-VH` are the highest, exceeding on all five targets, whereas the time overheads of `dipri-VE` are the lowest among all six configurations. Second, the four

Table 10. Summary of the Crashes Found by Zest and the Six DIPRI Configurations on the Java Benchmark

Statistics of Total Counts								
Target	Bug	zest	dipri-AE	dipri-AH	dipri-PE	dipri-PH	dipri-VE	dipri-VH
ant	ISE	20	20	20	20	20	0	18
bcel	CFE	20	20	20	20	20	17	20
	AVE	30	30	25	30	29	9	25
closure	NPE	40353	48398	47432	179	7617	57	12288
	RE	31	23	25	0	3	0	4
	ISE	1	0	0	0	0	0	0
rhino	ISE	24107	29648	27471	738	4541	70	14015
	VE	10	10	10	8	10	1	10
	NPE	67	72	101	11	21	4	31
	CCE	20	22	37	0	3	1	14
	AIOOB	1	2	0	0	9	0	0
Statistics of Consistency								
Target	Bug	zest	dipri-AE	dipri-AH	dipri-PE	dipri-PH	dipri-VE	dipri-VH
ant	ISE	1.0	1.0	1.0	1.0	1.0	0.0	1.0
bcel	CFE	1.0	1.0	1.0	1.0	1.0	1.0	1.0
	AVE	1.0	1.0	1.0	1.0	1.0	0.4	0.9
closure	NPE	1.0	1.0	1.0	1.0	1.0	0.8	1.0
	RE	1.0	0.9	1.0	0.0	0.2	0.0	0.3
	ISE	0.1	0.0	0.0	0.0	0.0	0.0	0.0
rhino	ISE	1.0	1.0	1.0	1.0	1.0	0.6	1.0
	VE	1.0	1.0	1.0	0.8	1.0	0.1	1.0
	NPE	1.0	1.0	1.0	0.7	1.0	0.3	1.0
	CCE	0.8	0.9	1.0	0.0	0.2	0.1	0.7
	AIOOB	0.1	0.1	0.0	0.0	0.9	0.0	0.0

The upper half of the table shows the total counts of the bugs that the studied approaches find in all 10 trials, whereas the lower half shows the statistics of finding bugs, which are calculated as “# of trials finding the bug/# of all trials.” The “Bug” column shows the abbreviations of the Java exceptions triggered by the studied approaches, where “ISE” is `IllegalStateException`, “CFE” is `ClassFormatException`, “AVE” is `AssertionViolatedException`, “NPE” is `NullPointerException`, “RE” is `RuntimeException`, “VE” is `VerifyError`, “CCE” is `ClassCastException`, and “AIOOE” is `ArrayIndexOutOfBoundsException`. CFE and AVE are from the `org.apache.bcel` package, and the other exceptions are all from the `java.lang` package.

configurations under ADAPTIVE and PERIODICAL have similar time overheads and tendencies; the tendencies of the time overheads are first quickly increasing and then slowly decreasing, which is semblable with the ones appearing on C/C++ targets. The reason `dipri-VE` occupies the least time is that it can hardly find interesting test inputs and activate seed distance calculation due to getting stuck at limited code regions, which is just like what we discuss in Section 6.1.2. In the meantime, as `dipri-VH` does not get stuck in limited code regions, it will frequently conduct seed distance calculation and thus incur considerable time overheads. The result that `dipri-VH` and `dipri-VE` have opposite time overheads implies that the impact of seed distance calculation is more significant on the Java benchmark compared to that on the C/C++ benchmark, which should be attributed to the nature of the underlying programming languages (i.e., C/C++ is typically considered faster than Java). Moreover, the differences between the performance of VANILLA and those of the other modes are more significant on the Java benchmark than on the C/C++ benchmark, which suggests that when with Java implementation, the seed distance calculation can occupy more time and thus can impair the performance of DIPRI more greatly.

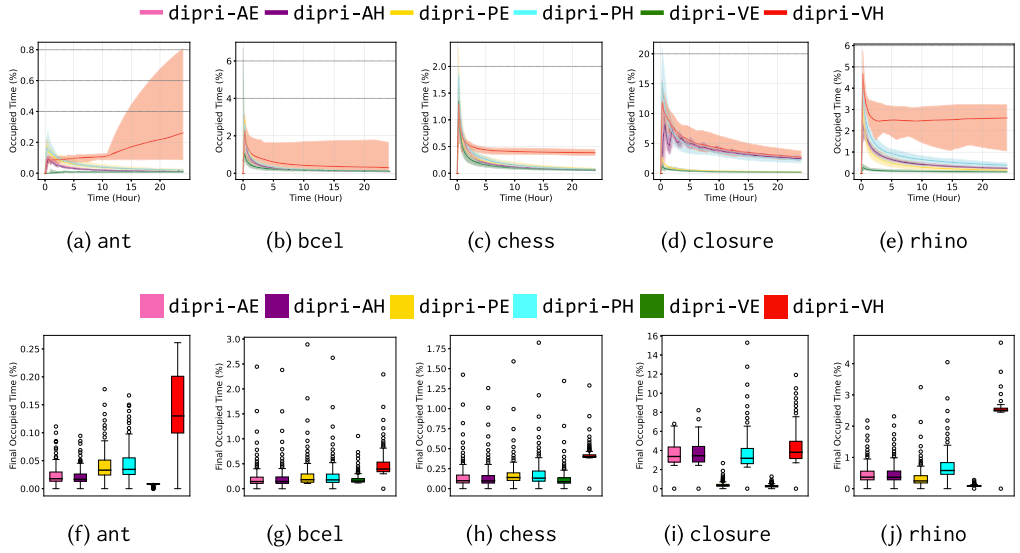


Fig. 18. The time occupancy of the six DiPRI configurations during 23-hour fuzzing. Parts (a) through (e) display the tendencies of time occupancy, whereas parts (f) through (j) display the distributions of final time occupancy.

6.2 Possible Extensions of DiPRI

In this article, we propose DiPRI, which to the best of our knowledge is the first distance-based seed prioritization approach to greybox fuzzing. We conduct well-designed experiments to empirically study not only the performance of the overall workflow of DiPRI but also the pros and cons of the different DiPRI configurations and individual subcomponents. Through our experiments, we find that the designs of DiPRI are far from perfect and have much room for improvement. Hoping to provide insights for follow-up researchers interested in this field, we organize the possible extensions of DiPRI as follows:

- *Adding strategies for seed picking and seed retirement:* In Section 5.1.4, we investigate how SPRs influence the performance of DiPRI under different configurations. We find that the performance of DiPRI can improve when the SPRs are improved. Similarly, we also observe that `af1++-Z`, which stands for the plainest order-of-queued seed prioritization approach, achieves the best rank on the eight fuzz targets from FuzzBench. These results all indicate that it may be improper to generate offspring inputs merely with the partial seeds prioritized at the forefront of the seed queue or simply retire the seeds ranked lower within the queue. Novel strategies are needed to customize the starting positions and ranges of picking seeds or choose the seeds to retire, such as designing heuristics according to the characteristics of the adopted seed evaluation metrics and introducing some machine learning based techniques.
- *Broadening the concept of seed distance:* In this work, we limit the concept of seed distance to coverage distance and explore how DiPRI performs on two different distance measures (i.e., Euclidean and Hamming). However, the coverage vector is just an incomplete indicator that can only profile partial natures of test inputs and test executions. For example, the content of a test input, or the data stored in a test input, is another important feature of test inputs. One can easily find two test inputs that achieve the same coverage vectors on the same PUT but contain different contents. Future works may consider constructing distances

among the contents of seeds, establishing correlations between the content distances and the performances of test inputs, and proposing novel techniques based on the established correlations.

- *Scaling to new fuzz targets*: Lots of research progress on fuzzing has emerged since this concept was formally introduced in the early 1990s [45, 50]. Recently, the improvements focusing fuzzing algorithms have shown less impact in practice and many researchers have switched their interests to applying fuzzing to targets from fresh scenarios [10, 23, 25, 61, 62]. DiPRI is a novel seed prioritization approach designed with simple motivation and general workflow. The integration of DiPRI on both the C/C++ fuzzer AFL++ and the Java fuzzer Zest has already demonstrated its good scalability. It will be interesting if we can apply DiPRI to more complex scenarios like REST API fuzzing, kernel fuzzing, and protocol fuzzing in the future.
- *Migrating to other steps of fuzzing*: The (mutation-based) greybox fuzzing relies on many steps to work, such as seed selection, seed prioritization, power scheduling, and input generation. Besides scaling to new fuzz targets, one can consider migrating the distance-based idea behind DiPRI to other steps of fuzzing. Promising directions include proposing a distance-based seed selection technique and designing a distance-based feedback mechanism. When constructing a distance-based technique, it will be beneficial if we can first identify the direction of distances, because the distance items, such as the outlier seeds prioritized by DiPRI, may accidentally steer fuzzing toward invalid code regions instead of unexplored valid code regions. The culprit of such an accident is the coverage vector, as it only provides very limited information about the test input and can hardly profile the importance of the code regions the test input reached. Therefore, to identify the direction of distances, one can consider partitioning the PUT into different code regions, analyzing the importance of each code region, and constructing refined coverage vectors by weighting the vector elements with the importance of code regions.

6.3 Threats to Validity

6.3.1 Threats to Internal Validity. The internal validity relates to the fairness and completeness of the experimental setups [6, 51]. To address the threats to internal validity, we follow prior works to prepare initial seeds for targets [39], configure the lengths of fuzz campaigns to obtain adequate raw data [14], and repeat each fuzz campaign for 10 trials to establish statistical significance. We also conduct configuration experiments to provide guidelines for the choices of prioritization modes and distance measures (Section 5.1), comparison experiments with SOTA seed prioritization approaches and seed scheduling techniques to show DiPRI’s competitiveness (Sections 5.2 and 5.3), and ablation experiments to independently assess the proposed distance-based seed evaluation (Section 5.4). The experimental setups of this work can greatly alleviate the threats to internal validity.

6.3.2 Threats to External Validity. The external validity is about whether the results of a study can be generalized [6]. To resolve the threats to external validity, we include four benchmarks (i.e., three C/C++ benchmarks and one Java benchmark) in our experiments and implement DiPRI on top of two mainstream fuzzers from different programming languages (i.e., C/C++ and Java). The used benchmarks are either composed of real-world programs widely adopted by prior works [39, 51, 54, 79] or standard benchmark projects highly recommended by fuzzing researchers [14, 28, 30, 49]. Our choices of benchmarks greatly consolidate the external validity of this work.

6.3.3 Threats in Approach Design and Implementation. Incomplete approach designs and certain implementation choices may also incur threats to the validity of a study [31]. In this article we further perfect the design of DiPRI based on the FUZZING'23 version of this work [56] by enriching the workflow of DiPRI and providing detailed formalization about seed evaluation and seed prioritization. Possible incompleteness of DiPRI includes the lack of a seed-picking range customization and a seed retirement strategy, which we discuss in Section 6.2 and leave as future work. As for implementation, we choose to prototype DiPRI on top of mainstream fuzzers like AFL++ and Zest, and strictly construct distance calculation and seed reordering according to well-established definitions and algorithms, which can resolve possible implementation threats to a large extent.

7 RELATED WORK

Improving Greybox Fuzzing. Greybox fuzzing is a well-established technique for automatic bug finding. Although powerful, it still has much room for facilitation in aspects like theory, implementation, and applied fuzz targets [7, 8, 57, 73]. Many works focus on improving greybox fuzzing by upgrading its components [35, 40, 72, 77], such as seed scheduling [66, 75], input generation [43, 78], execution guidance [39, 40], and power scheduling [12, 79]. Seeds are critical for greybox fuzzing, as they shape the input generated at runtime. Therefore, some fields like seed selection and seed prioritization are dedicated to ensuring the quality of seeds either before fuzzing starts or ahead of input generation. In particular, seed selection aims at preparing high-quality seed corpora for greybox fuzzing. For example, Rebert et al. [59] crawl seeds from the web and introduce heuristics to refine them for formal fuzzing; Herrera et al. [30] conduct large-scale experiments to investigate how initial seeds impact the performance of greybox fuzzers and propose a constraint-based corpus minimization tool named *OptiMin*. In contrast to their works, we focus on the quality of the seeds at the runtime of fuzzing and propose to pick outlier seeds for input generation in priority. Seed scheduling is a technique similar to seed prioritization, which is also interested in choosing which seeds for the next fuzzing iteration. As for seed scheduling, She et al. [66] propose a novel scheduler named *K-Scheduler* to identify, prioritize, and optimize seeds at the horizons of explored and unexplored code regions. Zhao et al. [80] model scheduling problem as a Monte Carlo Tree Search problem and propose *Alphuzz* to prioritize seeds with respect to the mutation relationships among them. Unlike *K-Scheduler* and *Alphuzz*, DiPRI leverages the discrepancies between seeds' coverage vectors to prioritize outlier seeds and does not require complex analysis of horizon seeds or mutation relationships. BELIEFUZZ [34] and T-SCHEDULER [42] are also SOTA seed scheduling techniques. We do not include them in the experiments, as they are parallel with our work.

Directed Greybox Fuzzing. Unlike general greybox fuzzing, the goal of **directed greybox fuzzing (DGF)** is to quickly reach and exhaustively exercise the target locations rather than purely enlarging code coverage [74, 81]. Although both DGF research and this work adopt the concept of seed distance, the essential definitions of seed distance are distinctly different. Specifically, in this work, seed distance is modeled as the discrepancies of coverage vectors among seeds. When serving for DGF, seed distance usually terms the distance of the generated seeds to the targeted locations [13], which is significantly different from our modeling. Recently, DGF has become a hot field where innovative works have been consistently emerging [16, 22, 35, 38, 64, 70, 81, 83]. For instance, Du et al. [22] facilitate the general process of DGF with the help of deviation basic blocks. Lee et al. [38] propose a constraint-based strategy to save the fuzzing time before exposing target crashes and improve the efficiency of DGF. In particular, Zheng et al. [81] are inspired by the practice of trawl fishing and propose a novel technique named *FishFuzz*, which can easily handle tens of thousands of targets without loss of precision. The authors hold the idea that, like trawl fishing, DGF can get benefits from covering a wide range of code regions first before starting to

dig the target locations formally. To this end, FishFuzz is equipped with (1) a multiphase queue culling algorithm powered by novel target-independent distance metrics to prioritize promising seed on demand, and (2) a target management component to dynamically rank targets and discard exhausted targets in time. The distance metric that FishFuzz uses for queue culling varies for different phases. Specifically, at the inter-function exploration phase, FishFuzz adopts a function-level metric to pick seeds closer to targets, whereas at the exploitation phase, FishFuzz exploits a multidistance function to hit profitable targets as much as possible. Unlike FishFuzz which involves lots of phases along with multiple goals of prioritization, DiPRI is much simpler, as it only focuses on prioritizing outlier seeds that are likely to give birth to inputs executing unexplored code regions.

Fuzzing toward Diversity. Besides expanding code coverage and exposing potential bugs, improving test diversity is another important theme of research on greybox fuzzing. Enlightened by ecological science, Böhme [11] constructs a novel statistic framework named STADS to profile the diversity achieved by fuzzing. Based on the STADS estimator, Böhme et al. [12] then propose a powerful fuzzer named ENTROPIC to maximize the achieved test diversity by rewarding the seeds that provide the most information entropy. In addition to Böhme’s team, there are many other researchers paying attention to the test diversity problem that lies in fuzzing. For instance, Menendez and Clark [48] propose HashFuzz to improve the diversity of test sets produced by greybox fuzzers. They achieve the goal by partitioning the input space via injecting the hash values into the PUTs and controlling the distributions of the test inputs generated by the fuzzers. Moreover, Nguyen and Grunske [51] emphasize the behavioral diversity of generated test inputs. They implement a novel technique named *BeDivFuzz* on top of JQF [53] to spread the generated inputs evenly across the input space. Unlike the preceding works, we highlight the potential of the outlier seeds and try to steer fuzzing toward diversified areas via an innovative distance-based prioritization.

Adaptive Random Testing. ART is a long-standing research topic that has been studied for decades [17, 19, 33]. It is first proposed to facilitate random testing with the intuition of selecting more diversified test cases [18, 36]. The classic application scenario of ART is the unit testing of small programs requiring numeric test inputs [17, 46]. Starting from small toy programs, researchers are dedicated to investigating how to apply ART to more complex scenarios such as web applications [63, 71], embedded systems [29], and simulation controllers [47]. In this article, we set sail from the key insight behind ART (i.e., choosing diverse tests from different regions) and propose a distance-based seed prioritization approach to facilitate greybox fuzzing. To the best of our knowledge, we are the first to apply ART to the seed prioritization of greybox fuzzing. Except for the novel approach, we also conduct large-scale experiments with 29 real-world fuzz targets to systematically evaluate the proposed approach. Our work is unique from previous ART research in this respect.

8 CONCLUSION

Seed prioritization is a critical step of greybox fuzzing that can influence the trend of fuzzing by determining the order of the seeds chosen for input generation. Inspired by the thoughts of ART, we propose a distance-based seed prioritization approach named DiPRI to facilitate greybox fuzzing. Based on the hypothesis that the outlier seeds, which are distant from the other seeds in the same seed queue, are more likely to give birth to the offspring test inputs that can execute unexplored code regions, we calculate seed distances with the coverage vectors of the seeds and prioritizes the seeds with larger distance-based scores. To evaluate DiPRI, we conduct well-designed experiments with 24 fuzz targets from three C/C++ benchmarks, where 8 are from real-world projects, 8 are from FuzzBench, and 8 are from Magma, and compare DiPRI to four SOTA seed prioritization

approaches and seed scheduling techniques. The results indicate that DiPRI can insignificantly influence the code-covering capability of the host fuzzer (i.e., AFL++ in our experiments) and can improve the bug-finding capability by triggering five more Magma bugs. Moreover, to investigate the scalability of DiPRI, we integrate DiPRI into a Java fuzzer named *Zest* and conduct an empirical study with 5 Java targets. The results on the Java benchmark demonstrate the good scalability of DiPRI. With the empirical results we got from two host fuzzers and 29 from four benchmarks and two program languages, we also discuss the interesting extensions that can be constructed on DiPRI. It is our hope that these in-depth discussions can provide insights for follow-up researchers and shed light on more works with distance-based views.

ACKNOWLEDGEMENTS

The authors would like to thank the anonymous reviewers for their insightful comments.

REFERENCES

- [1] AFL. n.d. American Fuzzy Lop GitHub Repository. Retrieved April 30, 2023 from <https://github.com/google/AFL>
- [2] AFL++Team. n.d. American Fuzzy Lop Plus Plus (AFL++). Retrieved April 30, 2023 from <https://github.com/AFLplusplus/AFLplusplus>
- [3] AFL++Team. n.d. Corpus Minimization for American Fuzzy Lop. Retrieved May 7, 2023 from <https://github.com/AFLplusplus/AFLplusplus/blob/stable/afl-cmin>
- [4] AFL++Team. n.d. AFL/AFL++ Test Cases. Retrieved November 16, 2023 from <https://github.com/AFLplusplus/AFLplusplus/tree/stable/testcases>
- [5] Alphuzz-Team. n.d. Alphuzz Artifacts. Retrieved December 15, 2023 from <https://github.com/zzyyrr/Alphuzz>
- [6] Apostolos Ampatzoglou, Stamatia Bibi, Paris Avgeriou, Marijn Verbeek, and Alexander Chatzigeorgiou. 2019. Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. *Information and Software Technology* 106 (2019), 201–230.
- [7] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. 2019. Nautilus: Fishing for deep bugs with grammars. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS '19)*.
- [8] Cornelius Aschermann, Sergej Schumilo, Ali Abbasi, and Thorsten Holz. 2020. Ijon: Exploring deep state spaces via fuzzing. In *Proceedings of the IEEE Symposium on Security and Privacy*.
- [9] Muhammad Ashfaq, Rubing Huang, and Michael Omari. 2020. FSCS-SIMD: An efficient implementation of fixed-size-candidate-set adaptive random testing using SIMD instructions. In *Proceedings of the 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE '20)*. IEEE, 277–288.
- [10] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. Restler: Stateful rest API fuzzing. In *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE '19)*. IEEE, 748–758.
- [11] Marcel Böhme. 2018. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology* 27, 2 (2018), 1–52.
- [12] Marcel Böhme, Valentin J. M. Manès, and Sang Kil Cha. 2020. Boosting fuzzer efficiency: An information theoretic perspective. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE '20)*. 678–689.
- [13] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. 2329–2344.
- [14] Marcel Böhme, László Szekeres, and Jonathan Metzman. 2022. On the reliability of coverage-based fuzzer benchmarking. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*. 1621–1633.
- [15] Hongxu Chen, Shengjian Guo, Yinxing Xue, Yulei Sui, Cen Zhang, Yuekang Li, Haijun Wang, and Yang Liu. 2020. MUZZ: Thread-aware grey-box fuzzing for effective bug hunting in multithreaded programs. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security '20)*. 2325–2342.
- [16] Hongxu Chen, Yinxing Xue, Yuekang Li, Bihuan Chen, Xiaofei Xie, Xiuheng Wu, and Yang Liu. 2018. Hawkeye: Towards a desired directed grey-box fuzzer. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. 2095–2108.
- [17] Jinfu Chen, Yiming Wu, Chengying Mao, Tsong Yueh Chen, and Haibo Chen. 2021. MMFC-ART: A fixed-size-candidate-set adaptive random testing approach based on the modified metric-memory tree. In *Proceedings of the 2021 IEEE 21st International Conference on Software Quality, Reliability, and Security (QRS '21)*. IEEE, 248–259.

- [18] Tsong Yueh Chen, Fei-Ching Kuo, Robert G. Merkel, and T. H. Tse. 2010. Adaptive random testing: The art of test case diversity. *Journal of Systems and Software* 83, 1 (2010), 60–66.
- [19] Tsong Yueh Chen, T. H. Tse, and Yuen-Tak Yu. 2001. Proportional sampling strategy: A compendium and some insights. *Journal of Systems and Software* 58, 1 (2001), 65–81.
- [20] Cliff Chow, Tsong Yueh Chen, and T. H. Tse. 2013. The ART of divide and conquer: An innovative approach to improving the efficiency of adaptive random testing. In *Proceedings of the 2013 13th International Conference on Quality Software*. IEEE, 268–275.
- [21] DiPri-Team. n.d. DiPri Artifacts. Retrieved January 15, 2024 from <https://github.com/QRXqrx/dipri-artifacts>
- [22] Zhengjie Du, Yuekang Li, Yang Liu, and Bing Mao. 2022. WindRanger: A directed greybox fuzzer driven by deviation basic blocks. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*. 2440–2451.
- [23] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, Surya Nepal, and Yang Xiang. 2021. Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 337–350.
- [24] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining incremental steps of fuzzing research. In *Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT '20)*.
- [25] Marius Fleischer, Dipanjan Das, Priyanka Bose, Weiheng Bai, Kangjie Lu, Mathias Payer, Christopher Kruegel, and Giovanni Vigna. 2023. ACTOR: Action-guided kernel fuzzing. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*. 5003–5020.
- [26] Google FuzzBench. n.d. List of the Fuzz Targets from FuzzBench. Retrieved December 21, 2023 from <https://google.github.io/fuzzbench/reference/benchmarks/>
- [27] Oded Goldreich. 2017. *Introduction to Property Testing*. Cambridge University Press.
- [28] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3 (2020), 1–29.
- [29] Hadi Hemmati, Andrea Arcuri, and Lionel Briand. 2013. Achieving scalable model-based testing through test case diversity. *ACM Transactions on Software Engineering and Methodology* 22, 1 (2013), 1–42.
- [30] Adrian Herrera, Hendra Gunadi, Shane Magrath, Michael Norrish, Mathias Payer, and Antony L. Hosking. 2021. Seed selection for successful fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 230–243.
- [31] Adrian Herrera, Mathias Payer, and Antony L. Hosking. 2023. DatAFLow: Toward a data-flow-guided fuzzer. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–31.
- [32] HexHive. n.d. Magma Bug List. Retrieved January 15, 2024 from <https://hexhive.epfl.ch/magma/docs/bugs.html>
- [33] Shan-Shan Hou, Chun Zhang, Dan Hao, and Lu Zhang. 2013. PathART: Path-sensitive adaptive random testing. In *Proceedings of the 5th Asia-Pacific Symposium on Internetware*. 1–4.
- [34] Heqing Huang, Hung-Chun Chiu, Qingkai Shi, Peisen Yao, and Charles Zhang. 2023. Balance seed scheduling via Monte Carlo planning. *IEEE Transactions on Dependable and Secure Computing*. Early Access, June 13, 2023.
- [35] Heqing Huang, Yiyuan Guo, Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2022. BEACON: Directed grey-box fuzzing with provable path pruning. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP '22)*.
- [36] Rubing Huang, Weifeng Sun, Yinyin Xu, Haibo Chen, Dave Towey, and Xin Xia. 2019. A survey on adaptive random testing. *IEEE Transactions on Software Engineering* 47, 10 (2019), 2052–2083.
- [37] Fei-Ching Kuo, Tsong Yueh Chen, Huai Liu, and Wing Kwong Chan. 2008. Enhancing adaptive random testing for programs with high dimensional input domains or failure-unrelated parameters. *Software Quality Journal* 16 (2008), 303–327.
- [38] Gwangmu Lee, Woonchul Shim, and Byoungyoung Lee. 2021. Constraint-guided directed greybox fuzzing. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*. 3559–3576.
- [39] Caroline Lemieux and Koushik Sen. 2018. FairFuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE '18)*. 475–485.
- [40] Shaohua Li and Zhendong Su. 2023. Accelerating fuzzing through prefix-guided execution. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 1–27.
- [41] LLVM.org. n.d. The llvm-cov Code Coverage Tool. Retrieved January 5, 2024 from <https://releases.llvm.org/14.0.0/docs/CommandGuide/llvm-cov.html>
- [42] Simon Luo, Adrian Herrera, Paul Quirk, Michael Chase, Damith C. Ranasinghe, and Salil S. Kanhere. 2023. Make out like a (Multi-Armed) Bandit: Improving the odds of fuzzer seed scheduling with T-Scheduler. *arXiv preprint arXiv:2312.04749* (2023).

- [43] Chenyang Lyu, Shouling Ji, Chao Zhang, Yuwei Li, Wei-Han Lee, Yu Song, and Raheem Beyah. 2019. MOPT: Optimized mutation scheduling for fuzzers. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security '19)*. 1949–1966.
- [44] Hexhive Magma. n.d. List of the Fuzz Targets from Magma. Accessed December 21, 2023 from <https://hexhive.epfl.ch/magma/>
- [45] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2019. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering* 47, 11 (2019), 2312–2331.
- [46] Chengying Mao, Xuzheng Zhan, T. H. Tse, and Tsong Yueh Chen. 2019. KDFC-ART: A KD-tree approach to enhancing fixed-size-candidate-set adaptive random testing. *IEEE Transactions on Reliability* 68, 4 (2019), 1444–1469.
- [47] Reza Matinnejad, Shiva Nejati, Lionel C. Briand, and Thomas Bruckmann. 2015. Effective test suites for mixed discrete-continuous stateflow controllers. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 84–95.
- [48] Hector D. Menendez and David Clark. 2021. Hashing fuzzing: Introducing input diversity to improve crash detection. *IEEE Transactions on Software Engineering* 48, 9 (2021), 3540–3553.
- [49] Jonathan Metzman, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. 2021. FuzzBench: An open fuzzer benchmarking platform and service. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE '21)*. 1393–1403.
- [50] Barton P. Miller, Louis Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Communications of the ACM* 33, 12 (1990), 32–44.
- [51] Hoang Lam Nguyen and Lars Grunske. 2022. BeDivFuzz: Integrating behavioral diversity into generator-based fuzzing. In *Proceedings of the 44th IEEE/ACM International Conference on Software Engineering (ICSE '22)*. 249–261.
- [52] Mohammad Norouzi, David J. Fleet, and Russ R. Salakhutdinov. 2012. Hamming distance metric learning. In *Advances in Neural Information Processing Systems*, Vol. 25. Curran Associates.
- [53] Rohan Padhye, Caroline Lemieux, and Koushik Sen. 2019. JQF: Coverage-guided property-based testing in Java. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 398–401.
- [54] Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019. Semantic fuzzing with zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '19)*. 329–340.
- [55] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2020. AFLNet: A greybox fuzzer for network protocols. In *Proceedings of the 2020 IEEE 13th International Conference on Software Testing, Validation, and Verification (ICST '20)*. IEEE, 460–465.
- [56] Ruixiang Qian, Quanjun Zhang, Chunrong Fang, and Zhenyu Chen. 2023. DiPri: Distance-based seed prioritization for greybox fuzzing (registered report). In *Proceedings of the 2nd International Fuzzing Workshop*. 21–30.
- [57] Ruixiang Qian, Quanjun Zhang, Chunrong Fang, and Lihua Guo. 2022. Investigating coverage guided fuzzing with mutation testing. In *Proceedings of the 13th Asia-Pacific Symposium on Internetwork*. 272–281.
- [58] Ruixiang Qian, Yuan Zhao, Duo Men, Yang Feng, Qingkai Shi, Yong Huang, and Zhenyu Chen. 2020. Test recommendation system based on slicing coverage filtering. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 573–576.
- [59] Alexandre Rebert, Sang Kil Cha, Thanassis Avgerinos, Jonathan Foote, David Warren, Gustavo Grieco, and David Brumley. 2014. Optimizing seed selection for fuzzing. In *Proceedings of the 23rd USENIX Security Symposium (USENIX Security '14)*. 861–875.
- [60] Christoph Schneckenburger and Johannes Mayer. 2007. Towards the determination of typical failure patterns. In *Proceedings of the 4th International Workshop on Software Quality Assurance: In Conjunction with the 6th ESEC/FSE Joint Meeting*. 90–93.
- [61] Sergej Schumilo, Cornelius Aschermann, Ali Abbasi, Simon Wörner, and Thorsten Holz. 2021. Nyx: Greybox hypervisor fuzzing using fast snapshots and affine types. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security '21)*. <https://www.usenix.org/conference/usenixsecurity21/presentation/schumilo>
- [62] Sergej Schumilo, Cornelius Aschermann, Andrea Jemmett, Ali Abbasi, and Thorsten Holz. 2022. Nyx-Net: Network fuzzing with incremental snapshots. In *Proceedings of the 17th European Conference on Computer Systems (Eurosys '22)*. <https://doi.org/10.1145/3492321.3519591>
- [63] Elmin Selay, Zhi Quan Zhou, and Jingjie Zou. 2014. Adaptive random testing for image comparison in regression web testing. In *Proceedings of the 2014 International Conference on Digital Image Computing: Techniques and Applications (DICTA '14)*. IEEE, 1–7.
- [64] Abhishek Shah, Dongdong She, Samanway Sadhu, Krish Singal, Peter Coffman, and Suman Jana. 2022. MC2: Rigorous and efficient directed greybox fuzzing. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2595–2609.

- [65] Dongdong She, Kexin Pei, Dave Epstein, Junfeng Yang, Baishakhi Ray, and Suman Jana. 2019. NEUZZ: Efficient fuzzing with neural program smoothing. In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP '19)*. 803–817.
- [66] Dongdong She, Abhishek Shah, and Suman Jana. 2022. Effective seed scheduling for fuzzing with graph centrality analysis. In *Proceedings of 2022 IEEE Symposium on Security and Privacy (SP '22)*.
- [67] Ji Shi, Zhun Wang, Zhiyao Feng, Yang Lan, Shisong Qin, Wei You, Wei Zou, Mathias Payer, and Chao Zhang. 2023. AIFORE: Smart fuzzing based on automatic input format reverse engineering. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*. 4967–4984.
- [68] Prashast Srivastava and Mathias Payer. 2021. Gramatron: Effective grammar-aware fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 244–256.
- [69] Weisong Sun, Quanjun Zhang, Chunrong Fang, Yuchen Chen, Xingya Wang, and Ziyuan Wang. 2022. Test case recommendation based on balanced distance of test targets. *Information and Software Technology* 150 (2022), 106994.
- [70] Xin Tan, Yuan Zhang, Jiadong Lu, Xin Xiong, Zhuang Liu, and Min Yang. 2023. SyzDirect: Directed greybox fuzzing for Linux kernel. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1630–1644.
- [71] Andrew F. Tappenden and James Miller. 2014. Automated cookie collection testing. *ACM Transactions on Software Engineering and Methodology* 23, 1 (2014), 1–40.
- [72] Erik Trickel, Fabio Pagani, Chang Zhu, Lukas Dresel, Giovanni Vigna, Christopher Kruegel, Ruoyu Wang, Tiffany Bao, Yan Shoshitaishvili, and Adam Doupe. 2022. Toss a fault to your Witcher: Applying grey-box coverage-guided mutational fuzzing to detect SQL and command injection vulnerabilities. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP '22)*. IEEE, 116–133.
- [73] Vasudev Vikram, Isabella Laybourn, Ao Li, Nicole Nair, Kelton OBrien, Raffaello Sanna, and Rohan Padhye. 2023. Guiding greybox fuzzing with mutation testing. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 929–941.
- [74] Pengfei Wang, Xu Zhou, Tai Yue, Peihong Lin, Yingying Liu, and Kai Lu. 2020. Sok: The progress, challenges, and perspectives of directed greybox fuzzing. *arXiv preprint arXiv:2005.11907* (2020).
- [75] Yanhao Wang, Xiangkun Jia, Yuwei Liu, Kyle Zeng, Tiffany Bao, Dinghao Wu, and Purui Su. 2020. Not all coverage measurements are equal: Fuzzing by coverage accounting for input prioritization. In *Proceedings of the Network and Distributed System Security Symposium (NDSS '20)*. 1–17.
- [76] Mingyuan Wu, Ling Jiang, Jiahong Xiang, Yanwei Huang, Heming Cui, Lingming Zhang, and Yuqun Zhang. 2022. One fuzzing strategy to rule them all. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. 1634–1645.
- [77] Wei-Cheng Wu, Bernard Nongpoh, Marwan Nour, Michaël Marcozzi, Sébastien Bardin, and Christophe Hauser. 2023. Fine-grained coverage-based fuzzing. *ACM Transactions on Software Engineering and Methodology*. Published Online, March 14, 2023.
- [78] Wei You, Xuwei Liu, Shiqing Ma, David Perry, Xiangyu Zhang, and Bin Liang. 2019. SLF: Fuzzing without valid seed inputs. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE '19)*. 712–723.
- [79] Tai Yue, Pengfei Wang, Yong Tang, Enze Wang, Bo Yu, Kai Lu, and Xu Zhou. 2020. EcoFuzz: Adaptive energy-saving greybox fuzzing as a variant of the adversarial multi-armed bandit. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security '20)*. 2307–2324.
- [80] Yiru Zhao, Xiaoke Wang, Lei Zhao, Yueqiang Cheng, and Heng Yin. 2022. Alphuzz: Monte Carlo search on seed-mutation tree for coverage-guided fuzzing. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 534–547.
- [81] Han Zheng, Jiayuan Zhang, Yuhang Huang, Zezhong Ren, He Wang, Chunjie Cao, Yuqing Zhang, Flavio Toffalini, and Mathias Payer. 2023. FISHFUZZ: Catch deeper bugs by throwing larger nets. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*. 1343–1360. <https://www.usenix.org/conference/usenixsecurity23/presentation/zheng>
- [82] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. 2022. Fuzzing: A survey for roadmap. *ACM Computing Surveys*. Published Online, September 9, 2022.
- [83] Peiyuan Zong, Tao Lv, Dawei Wang, Zizhuang Deng, Ruigang Liang, and Kai Chen. 2020. FuzzGuard: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security '20)*. 2255–2269.

Received 26 July 2023; revised 31 January 2024; accepted 14 March 2024